Airbnb Senior ML Engineer

Onsite interview question bank

01. Loop Format and the Senior Bar: Airbnb Senior MLE (Experimentation / A/B Testing)
- Format notes
- Questions
- References

02. Coding Rounds: Airbnb-Tagged LeetCode and Reported Practical Coding Questions
- Format notes
- Questions
- References

03. ML System Design: Search Ranking, Embeddings, Pricing, Trust, Recommendations and More
- Format notes
- Questions
- References

04. A/B Testing and Experimentation Deep Dive: ERF, Marketplace Interference, Variance Reduction, Interleaving and Sequential Testing
- Format notes
- Questions
- References

05. ML Fundamentals and Modeling Depth: Ranking, GBDT vs. NN, Embeddings, Calibration, Features, Offline/Online Gap
- Format notes
- Questions
- References

06. General Backend System Design: Booking, Availability, Search, Messaging, Payments and Feature Store
- Format notes
- Questions
- References

07. Past Project / Technical Experience Deep Dive: Reported Prompts and Probing Follow-ups for Senior MLEs
- Format notes
- Questions
- References

08. Core Values Interviews: Champion the Mission, Be a Host, Embrace the Adventure, Be a Cereal Entrepreneur, plus Cross-Functional and Leadership Behavioral Questions
- Format notes
- Questions
- References

01. Loop Format and the Senior Bar: Airbnb Senior MLE (Experimentation / A/B Testing)

> Research note (read first). Live web research was not possible when this chapter was written. The shared web-search budget was used up, and the egress proxy refused connections to igotanoffer.com, interviewing.io, careers.airbnb.com and airbnb.tech. So this chapter draws on what was already known about Airbnb's widely reported interview process and on the sources listed under References. None of those sources were re-fetched for this chapter. Before relying on a URL or a "[Reported]" tag, open the source and confirm it. A recruiter's prep packet always overrides this chapter. Tags: [Reported] means a public source or the LeetCode Airbnb company tag attributes the item to Airbnb. [Likely] means an inference from the role, the level and Airbnb's products. It does not mean anyone reported it.

Format notes

Stage	Typical length	Interviewer	What is assessed (Senior / L5 bar)
Recruiter call	20-30 min	Recruiter	Motivation ("Why Airbnb?"), level calibration (L5 = Senior), team fit (Search/Ranking, Pricing, Trust, Payments, Relevance, Experimentation platform), comp, timeline [1][2]
Technical phone screen	45-60 min	Engineer	One or two practical coding problems on a shared editor (CoderPad-style). The code is expected to **compile and run against test cases**. Pseudocode does not pass [1][2][3]
Onsite: Coding x2	45-60 min each	Engineers	Runnable, clean, tested code. One round is often algorithmic (LeetCode-medium/hard, many Airbnb-tagged problems). The other is a longer "practical" problem with evolving requirements (parsing, pagination, scheduling, simulation) [1][3][4]
Onsite: ML design	45-60 min	Senior/Staff MLE	End-to-end ML system for an Airbnb product (search ranking, pricing, fraud, recommendations), including offline/online evaluation and **experiment design** [Likely; 6-10]
Onsite: System design	45-60 min	Senior/Staff engineer	Scalable services and data pipelines (for example a booking system, an experimentation/assignment service, a feature store) [1][2]
Onsite: Past project deep dive	45-60 min	Hiring manager or senior IC	Ownership, technical depth, trade-offs and measured impact of a project you led. Sometimes presented with slides [1][2]
Onsite: Core values x2	30-45 min each	Employees **outside** the hiring team (often non-engineers)	Airbnb's four core values: *Champion the Mission, Be a Host, Embrace the Adventure, Be a "Cereal" Entrepreneur*. A strong fail here can veto an otherwise strong loop [1][2][5]

Stage	Typical length	Interviewer	What is assessed (Senior / L5 bar)
Debrief / hiring committee	n/a	Panel + leadership review	Calibrated on level. Senior candidates are expected to drive ambiguity, lead cross-functional work and show measured impact [1][2]

Key expectations that candidates consistently report:

- **Runnable code is mandatory.** Interviewers run your code. Syntax errors, unhandled edge cases or no tests count against you even if the logic is right [1][3].
- **Practical over puzzle.** Problems are often framed as real product scenarios: listings pagination, host search results, price rounding, travel itineraries, calendar availability [3][4].
- **Core values are scored as seriously as the technical rounds** and are run by people with no stake in the team [1][5].
- **Senior bar (L5):** you independently own an ambiguous problem end to end. You make clear trade-offs (latency vs. quality, bias vs. variance in experiment estimates). You influence partners such as Product, Data Science and Design, and you mentor others. Coding and design are expected to be done without heavy hints [Likely; 1][2].
- **Experimentation-flavored MLE roles:** Airbnb runs a mature in-house experimentation platform (ERF) and has written publicly about A/B testing pitfalls, interleaving for search ranking and two-sided-marketplace effects. Expect experimentation probes inside the ML design and past-project rounds [6][7][8][9].

Questions

A. Recruiter call and process logistics

1. Why Airbnb, and why now? [Reported] [1][2]
    - Which Airbnb product surface (Search, Pricing, Trust, Experiences, Payments) would you most want to work on, and why?
    - Have you used Airbnb as a guest or as a host? What would you change?
2. Walk me through your background in under three minutes, focusing on ML in production. [Reported] [1]
3. What level are you targeting, and what scope of work did you own at your current level? [Likely]
    - How many engineers or scientists did you lead or influence on your largest project?
4. Which programming language will you use in coding rounds, and are you fluent enough to run code without IDE help? [Likely] [3]
5. How much of your recent work was modeling, and how much was infrastructure or experimentation? [Likely]
6. Have you designed or analyzed online A/B tests yourself, or did a data scientist own that? [Likely]
7. What are your compensation expectations and timeline, and do you have competing processes? [Reported] [2]

8. Are you open to team matching across Search/Relevance, Pricing, Trust and the Experimentation platform? [Likely]

B. Technical phone screen (practical coding, must run)

9. Given a list of listings (host_id, listing_id, score, city) and a page size, return paginated results so that no host appears twice on the same page if possible, preserving score order. [Reported] [3][4]
    - What happens on the last page when only one host's listings remain?
    - What is the time complexity, and can you do it in a single pass?
10. Minimize rounding error: round a list of prices to integers so the sum equals a target, minimizing the total absolute rounding error. [Reported] [4]
    - What if the target is unreachable? How do you signal that?
    - How would you handle currency with two decimal places?
11. Pour water: simulate water droplets falling onto an elevation map at index K, with the leftward-first flow rule. [Reported] [4]
    - How would you print the final terrain and water visually?
    - How do you test the boundary indices?
12. Text justification: fully justify lines of words to a fixed width. [Reported] [4]
    - How do you handle a single word on a line, and the last line?
13. Flatten a 2D vector into an iterator with `next`, `hasNext` and (as a follow-up) `remove`. [Reported] [4]
14. Implement a CSV parser that handles quoted fields, escaped quotes and commas inside quotes. [Reported] [3]
    - How do you handle multi-line quoted fields?
15. Given an IP address and a count, return the minimal list of CIDR blocks covering exactly that range. [Reported] [4]
16. Write tests for your solution right now. Which cases did you choose, and why? [Likely] [3]

C. Onsite coding round 1 (algorithmic, Airbnb-tagged style)

17. Alien dictionary: derive a character ordering from a sorted list of words. [Reported] [4]
    - How do you detect an invalid ordering, such as a prefix appearing after a longer word?
18. Palindrome pairs: find all index pairs whose concatenation forms a palindrome. [Reported] [4]
    - Can you do better than O(n^2 * k)?
19. Cheapest flights within K stops (framed as a travel-itinerary cost minimizer). [Reported] [4]
    - What changes if edges carry both price and duration and you must optimize a weighted objective?
20. Sliding puzzle: minimum moves to solve a 2x3 board. [Reported] [4]
21. Pyramid transition: can you build a pyramid from a bottom row given allowed triples? [Reported] [4]
22. Employee free time: given each person's busy intervals, return the common free intervals (calendar availability). [Reported] [4]

    - How would you adapt this to host calendars with blocked dates and minimum-night rules?
23. Smallest common region: find the lowest common region in a region hierarchy. [Reported] [4]
24. Combination sum (framed as "choose menu items that exactly use a budget"). [Reported] [4]
    - How do you handle float prices without precision bugs?
25. Regular expression matching with `.` and `*`. [Reported] [4]
26. Word search II on a Boggle-style board. [Reported] [4]
27. Basic calculator with `+ - * /` and parentheses. [Reported] [4]
28. Contains duplicate within distance k and within value difference t. [Reported] [4]
29. Flatten a nested list iterator / mini parser for nested lists. [Reported] [4]
30. House robber (framed as non-adjacent booking selection maximizing revenue). [Reported] [4]
    - What if bookings are intervals with arbitrary overlaps (weighted interval scheduling)?

D. Onsite coding round 2 (practical, evolving requirements)

31. Design and implement a booking calendar that supports `book(start, end)`, rejects overlaps and answers `is_available(start, end)`. [Likely] [3]
    - Now add per-night pricing and return the total price for a stay.
    - Now support split stays across two listings when no single listing is available.
32. Implement a simple key-value store with transactions (`begin`, `commit`, `rollback`), nested. [Likely] [3]
33. Parse a log of experiment exposures (user_id, experiment, variant, timestamp) and output per-variant unique user counts, de-duplicating users exposed to multiple variants. [Likely]
    - How would you flag and count users with conflicting assignments?
34. Implement deterministic hash-based bucketing: given user_id, experiment salt and traffic split, return a variant. [Likely] [6]
    - How do you guarantee independence between experiments run on the same users?
    - How would you ramp traffic from 1% to 50% without reshuffling already-exposed users?
35. Given streaming events, compute a rolling 7-day conversion rate per listing with bounded memory. [Likely]
36. Implement a file system with `create(path, value)` and `get(path)`, including parent-existence checks. [Reported] [4]
37. Given guest reviews with ratings and timestamps, return the top-K listings by a time-decayed average rating. [Likely]
    - How do you handle listings with very few reviews (shrinkage)?
38. Implement an LRU or LFU cache for search results, then extend it with a TTL. [Likely]
39. Refactor your solution so that a new requirement (for example multi-currency support) fits in with minimal change. How do you structure the code? [Likely] [3]

E. ML design round (Airbnb product + experimentation)

40. Design the search ranking system for Airbnb stays. [Likely] [7][8][9]
    - What are your labels (bookings, clicks, long clicks, host acceptances), and how do you weight them?
    - How do you handle position bias in training data?
    - How do you evaluate offline (NDCG on booked listings) versus online?
41. Design real-time personalization using listing embeddings from in-session clicks. [Likely] [8]
    - How would you inject booking or market signals into the embedding objective?
    - How do you handle cold-start listings with no sessions?
42. Design a dynamic pricing suggestion model for hosts (Smart Pricing). [Likely]
    - How do you estimate demand elasticity when price is confounded with seasonality and quality?
    - How do you A/B test a pricing model when hosts can override it?
43. Design a fraud or risk model for bookings (Trust and Safety). [Likely]
    - How do you evaluate a model whose positive labels arrive weeks late and are censored by your own interventions?
44. Design "similar listings" or "recommended experiences" recommendations. [Likely]
45. Design a model that predicts host acceptance probability and use it in ranking. [Likely] [7]
46. Design an ML system to detect low-quality or misleading listing photos and text. [Likely]
47. How would you design the online experiment that launches your new ranking model? [Likely] [6][9]
    - What is the primary metric, and what are your guardrail metrics?
    - What unit of randomization would you use (user, session, listing, market), and why?
    - How do you estimate the required sample size and duration?
48. In a two-sided marketplace, why can a user-randomized test of a ranking change be biased? [Likely] [6]
    - How would listing-side or market-level (cluster) randomization change your estimates?
    - What is interference, or a SUTVA violation, in this setting?
49. When would you use interleaving instead of A/B testing for search ranking? [Likely] [9]
    - How does interleaving sensitivity compare with A/B, and what can interleaving not measure?
50. Offline NDCG improved but the online A/B showed no booking lift. How do you debug this? [Likely] [7]
51. Bookings happen weeks after search. How do you choose an experiment metric that is sensitive enough but still aligned with long-term value? [Likely] [6]
    - How would you build and validate a proxy or surrogate metric?
52. How would you apply variance reduction (CUPED or regression adjustment) to booking-conversion experiments? [Likely] [6]
53. How do you handle novelty effects and long-tail seasonality (holidays, events) when reading test results? [Likely]
54. How do you detect and correct sample ratio mismatch? [Likely] [6]
55. How do you control false discoveries when a team runs dozens of concurrent experiments and checks results daily? [Likely] [6]

    - What is the peeking problem, and how do sequential tests address it?
56. How would you measure the long-term effect of a model change after the A/B test has ended (holdouts)? [Likely]
57. Your model improves guest conversion but reduces host earnings in some markets. How do you decide whether to ship? [Likely]

F. System design round

58. Design an experimentation assignment and logging service used by every Airbnb client (web, iOS, Android, backend). [Likely] [6]
    - How do you keep assignment consistent across logged-out and logged-in states?
    - How do you guarantee exposure logging is not lost or double-counted?
59. Design the metrics computation pipeline for an experimentation platform (daily scorecards for thousands of metrics). [Likely] [6]
    - How do you backfill when a metric definition changes?
60. Design a booking and reservation system that prevents double-booking under high concurrency. [Likely] [1]
61. Design a feature store that serves both offline training and low-latency online ranking. [Likely]
    - How do you prevent training-serving skew?
62. Design a search service that retrieves and ranks listings within 200 ms p99 across millions of listings. [Likely] [7]
    - Where do you place candidate retrieval, filtering (availability, price, map bounds) and ML re-ranking?
63. Design a model serving and rollout system with shadow mode, canary and automatic rollback. [Likely]
64. Design a notification system for hosts and guests that respects time zones and rate limits. [Likely]
65. Design a calendar availability service for millions of listings with frequent updates and iCal sync. [Likely]

G. Past project deep dive

66. Walk me through the ML project you are proudest of, from problem framing to measured impact. [Reported] [1][2]
    - What was your specific contribution compared with the team's?
    - What alternatives did you reject, and why?
67. How did you measure the impact of that project? Was there an A/B test, and what did it show? [Likely]
    - What were the confidence intervals, and were there any surprising segment-level results?
68. What was the hardest technical trade-off you made on that project? [Likely]
69. If you had to redo the project with half the time, what would you cut? [Likely]
70. Tell me about a model that worked offline but failed in production. What did you learn? [Likely]
71. How did you get alignment from Product and Data Science on the success metric? [Likely]
72. What would the next iteration of that system look like? [Likely]

73. How did you handle data quality or labeling problems in that project? [Likely]
74. Which part of the system would break first at 10x scale? [Likely]

H. Core values interviews (two rounds, cross-functional)

75. Which of Airbnb's core values resonates with you most, and why? [Reported] [1][5]
76. Tell me about a time you went above and beyond for a customer, user or teammate ("Be a Host"). [Reported] [1][5]
77. Tell me about a time you took a big risk or tried something no one had done before ("Embrace the Adventure"). [Reported] [1][5]
78. Tell me about a time you had to be scrappy and resourceful with limited resources ("Be a Cereal Entrepreneur"). [Reported] [1][5]
79. What does "belonging" mean to you, and when have you created it for someone else ("Champion the Mission")? [Reported] [1][5]
80. Describe a memorable travel or hosting experience and what it taught you. [Reported] [1]
81. Tell me about a time you failed. What did you do next? [Reported] [1][2]
82. Tell me about a time you disagreed with a teammate or manager. How did you resolve it? [Reported] [2]
83. Tell me about a time you received difficult feedback. [Reported] [2]
84. Describe a time you helped someone grow or mentored a junior colleague. [Likely]
85. Tell me about a time you made a decision that was right for users but unpopular internally. [Likely]
86. How do you build inclusive teams or include people with different perspectives? [Likely] [5]
87. Tell me about a time you had to adapt to a big change you did not choose (a reorg, layoffs, a pivot). [Likely]
88. What would your previous teammates say is the one thing you should improve? [Likely]
89. Why do you want to work on a two-sided marketplace product? [Likely]

I. Senior-level calibration probes (appear across rounds)

90. How do you decide when a simpler heuristic should ship instead of an ML model? [Likely]
91. Tell me about a time you set the technical direction for a team without formal authority. [Likely]
92. How do you prioritize between model quality, latency and engineering cost? [Likely]
93. Describe a time you stopped or reversed a launch because the experiment results were ambiguous. [Likely]
94. How do you communicate a negative or inconclusive A/B result to leadership? [Likely]
95. How do you make sure your experiment results are reproducible and trusted by other teams? [Likely]
96. What metrics would you put on a dashboard to know your production model is healthy? [Likely]
97. How do you balance short-term booking metrics against long-term guest and host trust? [Likely]

98. What questions do you have for us about the team, the experimentation culture or the roadmap? [Reported] [1]

References

The sources below were **not re-fetched** for this chapter because live web access was blocked. Each is listed because it was recalled from earlier knowledge as relevant. Confirm that each URL loads before you rely on it.

1. "Airbnb Interview Process & Questions (+ how to prepare)", IGotAnOffer, 2023-2024. https://www.igotanoffer.com/blogs/tech/airbnb-interview-process
2. "Airbnb Machine Learning Engineer Interview Questions", Glassdoor (candidate reports), ongoing. https://www.glassdoor.com/Interview/Airbnb-Machine-Learning-Engineer-Interview-Questions-EI_IE391850.0,6_KO7,32.htm
3. "Airbnb Interview Discussions", LeetCode Discuss (search tag "Airbnb"), ongoing. https://leetcode.com/discuss/interview-question?currentPage=1&orderBy=hot&query=airbnb
4. "Airbnb company-tagged problems", LeetCode (Premium company tag list), ongoing. https://leetcode.com/company/airbnb/
5. "Airbnb Careers: Life at Airbnb / core values", Airbnb, ongoing. https://careers.airbnb.com/
6. "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
7. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD 2019. https://arxiv.org/abs/1810.09591
8. Grbovic & Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
9. Haldar et al., "Improving Deep Learning for Airbnb Search", arXiv / KDD 2020. https://arxiv.org/abs/2002.05515
10. "The Airbnb Tech Blog", Airbnb Engineering (Medium), ongoing. https://medium.com/airbnb-engineering

02. Coding Rounds: Airbnb-Tagged LeetCode and Reported Practical Coding Questions

> Research note (read first). This chapter was researched on 2026-10-13. The shared web-search budget was already used up, and the network proxy could not reach Glassdoor, Blind, LeetCode, Reddit, 1point3acres, interviewing.io, Prepfully, IGotAnOffer or Airbnb's own sites. **GitHub was the only reachable source.** The evidence below therefore comes from pages fetched and read on GitHub:
>
> - two open mirrors of LeetCode's Airbnb company tag [5][6];
> - two mirrored candidate-report guides from TrueInterview, one for the MLE track and one for the SWE track [1][2];
> - a dated question tracker and a 2025-2026 company guide [3][4];
> - classic Airbnb question collections [7][8][9][10];
> - a GitHub mirror of about 100 individual 1point3acres (一亩三分地) Airbnb interview posts [13]. Posts that carry a specific question are cited individually [14]-[57].
>
> All of these are third-party, user-submitted material, so treat every [Reported] tag as "a candidate or aggregator says so", not as Airbnb confirmation. [Likely] marks an inference from the MLE/experimentation role. It does not mean anyone reported it.

Format notes

- **How many coding rounds an MLE gets.** The one public MLE-track summary (3 candidate reports, mainly 2026 Q2) gives MLEs one technical phone screen and **typically one onsite coding round** (45-60 min). It says the track is "filtered by design depth, not by code": the decisive rounds are 1-2 ML system design rounds [1]. One 1point3acres MLE-type loop had one coding round (LC 1298) plus two ML design rounds (LTV and host pricing) [14]. Another had two ML design rounds plus a "ConnectN" OOD coding round [15]. SWE-track loops usually have 1-2 coding rounds plus a separate **code review** round (about 3 PRs) [2][4]. Ask your recruiter whether the MLE loop includes code review.
- **Runnable code with tests is the bar.** Candidates report that Airbnb requires "bug free" code on the spot with all cases passing [20]. Other reports: code must run and be tested within 45 minutes [2]; there were "about 10 test cases" for Connect 4 [32]; problems arrived with stdin-style test strings [17]; one interviewer gave no interface, so the candidate had to agree the API and then write tests that pass [36]. Airbnb has long stressed "compilable, runnable code" [11]. One MLE screen expected the candidate to explain pruning or memoization *before* coding, then write test cases [1].

- **Shape of a round.** A round is one practical problem in 45 min, often in two parts. Interviewers add constraints once part 1 runs, so leave 10-15 minutes for follow-ups [4][33]. Several posts say the interviewer said nothing and just pasted the problem [18][20]. Communication still counts: a candidate who stayed silent was marked down [12]. Switching algorithms mid-round and polishing helpers too early are cited as reasons for rejection [2].
- **Flavor.** Problems are framed as product stories: split stays, group lodging, menus and combos, refunds, layovers, ski runs, host pagination, review highlighting. They test DP (often bitmask), graph search, simulation, parsing and small-object design rather than exotic tricks [1][3][4]. LeetCode's Airbnb tag over the last 6 months is led by Design Excel Sum Formula, Maximum Profit in Job Scheduling, Shortest Uncommon Substring, Maximum Candies from Boxes, Shopping Offers, Accounts Merge, Text Justification and Smallest Sufficient Team [5].
- **Senior bar.** Seniors are expected to clarify ambiguous specs themselves, for example tie-breaking rules, whether the graph has cycles, and whether to return the path or only the cost [16]. They must also reach working code fast enough to discuss complexity and extensions. One candidate passed with a suboptimal solution that ran and passed every test, and only *described* the optimization [12][19]. Another failed because they could not beat O(N^2) when asked [19]. Several senior rejections were attributed to design or deep-dive rounds even when coding was strong [16][28].

Questions

A. Airbnb "classic" practical problems (long-running question bank)

1. Display Page / pagination: given CSV search results (host_id, listing_id, score, city) sorted by score and a page size, reorder them so that a host appears at most once per page when possible, while otherwise preserving order. Print the pages as blocks. [Reported] [7][8][52]
    - What should happen when the remaining results all belong to one host?
    - Can you do it in one pass without repeatedly rescanning the list?
    - How would you test that the relative order inside a page is preserved?
2. Text Justification: pack words greedily into lines of exactly width L, spreading extra spaces left-first, with the last line left-justified. [Reported] [5][7][8]
    - How do you handle a line that holds a single word?
    - What changes if words can be longer than L?
3. Fixed-width text formatting (simpler variant: left-align and pad with trailing spaces), then lay the lines out into k side-by-side columns separated by one space. [Reported] [1][40][50]
    - When one column runs out of lines, what do you print in its place?
    - What is the complexity for n up to 1e5 words?
4. Menu combination sum: given menu item prices, return every combination of items whose total is exactly the budget. Items may repeat, and duplicate combinations are not allowed. [Reported] [7][49]
    - Prices are floats like 2.40. How do you avoid floating-point equality bugs? [Reported] [17]

    - Now return only the count of combinations. Does order matter? [Reported] [10] [48]
5. Palindrome Pairs: find all pairs (i, j) such that words[i] + words[j] is a palindrome. [Reported] [5][7][39]
    - The interviewer wants O(nk) rather than O(nk^2). Which requirement do you clarify first? [Reported] [16]
    - How do the empty string and duplicate words affect correctness?
6. IP range to CIDR: given a starting IPv4 address and a count n, return the shortest list of CIDR blocks that covers exactly that range. [Reported] [5][7][8]
    - How do you test the block boundaries, for example a range starting at an odd address?
7. List-of-list (2D) iterator with `hasNext`, `next` and a `remove` that may be called once per `next`. [Reported] [5][7][8]
    - What should `remove` do when it is called twice, or before `next`?
    - How would you make the iterator lazy over a very large nested input?
8. Flatten Nested List Iterator. [Reported] [5][6]
9. Cheapest flights / minimum cost with at most K stops. Return both the cost **and** the route. [Reported] [5][7][27]
    - If the route must be returned, which algorithms still work?
    - How would you handle ties, or a cycle of zero-cost edges?
10. Sliding puzzle: find the minimum moves to solve the 2x3 board (LC 773), or produce a move sequence for the 3x3 (8-puzzle) variant. [Reported] [5][7][8]
    - How do you detect that a board is unsolvable before searching?
    - Would A* help, and with which heuristic?
11. Pour water / water drop: given terrain heights, pour V units at index K and return the final heights. [Reported] [5][7]
12. Terrain rendering plus water simulation: Part 1 prints the terrain as ASCII from a heights array. Part 2 drops water at a column and prints the terrain with W for water. [Reported] [3][35]
    - Which way should water flow when both sides are equally low?
    - How do you test the printed output exactly, including trailing whitespace?
13. Round prices: round each price up or down so that the rounded values sum to the rounded total, minimizing the total rounding error (LC 1058). [Reported] [5][7][10]
    - What do you return when the target is unreachable?
14. CSV parser: parse a CSV line with quoted fields, escaped quotes and commas inside quotes. [Reported] [7][8]
    - How do you handle a quoted field that spans multiple lines?
15. Design File System: `create(path, value)` fails if the parent is missing, and `get(path)` returns the value. [Reported] [5][7]
    - How would you add `delete` with recursive children?
16. Alien Dictionary: derive the letter order from a sorted word list. [Reported] [5][6][8]
    - How do you detect an invalid input, such as a longer word appearing before its own prefix?
17. Regular expression matching with `.`, `*` and also `+`. [Reported] [5][7]
18. Boggle game: find the path on a letter grid that spells the **largest number** of dictionary words (not just any word). Word Search II is the tagged LeetCode version. [Reported] [5][7]

19. Pyramid transition matrix: can the bottom row be built up to a single root using the allowed triples? [Reported] [5][7][9]
20. Ten wizards: the cost of a link is the squared difference of wizard IDs. Find the minimum-cost path from wizard 0 to wizard 9. [Reported] [7][8][9]
21. Travel buddy list: friends whose wishlists overlap more than half with yours are "buddies". Recommend up to `max` cities from buddies' wishlists, ordered by similarity. [Reported] [7][57]
    - How do you break ties between buddies with equal overlap?
22. Collatz conjecture: find the longest step count for any start value in [1, n]. [Reported] [3][7]
    - How would you memoize, and how do you bound memory for large n?
23. Find the median of a file of integers too large for memory, with only sequential access. [Reported] [3][7][8]
24. Guess the number: find a hidden 4-digit code (digits 1-6) using an API that returns how many digits are in the correct position. Minimize the number of API calls. [Reported] [3][7][54]
    - Variant: talk to a server over a socket with a START/GUESS protocol. How do you structure the I/O? [Reported] [54]
25. Meeting time / Employee Free Time: return the common free intervals across everyone's schedules. [Reported] [5][7]
26. Maximum number of nights you can accommodate: choose non-adjacent booking requests to maximize nights (House Robber framing). [Reported] [5][7][9]
27. Generate all upper- and lower-case combinations of a string and test each against a hidden string. [Reported] [7]
28. K edit distance: find all dictionary words within edit distance k of a target. [Reported] [7][8]
    - How would a trie change the complexity?
29. Preference list: merge everyone's ranked lists into one ordering that respects every person's order. [Reported] [7][8]
30. Minimum vertices from which every node of a directed graph is reachable. [Reported] [3][5][7]
    - Does the graph contain cycles, and if so, how does that change the answer? [Reported] [16]
31. Finding ocean (flood fill from a water cell), number of intersecting rectangles, Hilbert curve step count, and Simulate Diplomacy (army move/hold/support resolution). [Reported] [7][8][10]
32. Implement a queue backed by a chain of fixed-size arrays. [Reported] [7][8]

B. Practical problems reported in 2025-2026 (current question bank)

33. Split stay: given listings with lists of available day numbers and a date range, return every pair of listings [X, Y] where X covers a prefix of the range and Y covers the rest (one split only). [Reported] [3][18][19][20]
    - Your O(N^2 * D) solution works. The interviewer says something better exists. Where is the redundant work?
    - What if a split stay may use up to three listings?
    - How do you deduplicate symmetric pairs, and must the order be preserved?

34. Group lodging / select properties: for a target neighborhood and group size, choose properties with total capacity >= group size, minimizing the number of properties and then the total capacity. Return the IDs, or -1 or [] if impossible. [Reported] [4][21] [41]
    - Which tie-break do you confirm first: fewer properties or less wasted capacity? [Reported] [21]
    - Is this subset-sum-like DP or backtracking, and what is the worst-case cost?
35. Minimum-cost menu order: given single items and combos with prices plus a list of wanted items (at most 3 unique), return the lowest cost **and** the items to order. [Reported] [1][16][22][23]
    - You wrote DFS. Now convert it to DP. How do you add bitmask state? [Reported] [22]
    - Which menu entries can you prune before searching? [Reported] [16][56]
    - Return every optimal combination, not just one. [Reported] [16]
    - Variant: rental "services" such as wifi, parking and TV instead of food. [Reported] [23]
36. Menu bundles closest to budget: choose up to 3 bundles whose total is closest to, but not above, the budget. Write at least 5 test cases. [Reported] [35]
37. Layover: the experience durations can each be booked repeatedly. Can you exactly fill the layover hours? Follow-up: what is the minimum number of bookings? [Reported] [1][3][51]
38. Connect Four simulation: take a list of column moves, stop on a win or an invalid move, ignore the remaining moves, and print the grid. [Reported] [3][17][32][37]
    - How do you avoid rescanning the column for gravity on every move? [Reported] [32]
    - Check the diagonals without off-by-one errors in the limited time. [Reported] [16]
    - Determine win, draw or ongoing from a static board alone. [Reported] [3]
39. ConnectN object model: configurable width, height and N in a row to win, alternating players, and draw detection. [Reported] [15]
    - Which classes and interfaces would you expose so that a new rule, such as an irregular board, is easy to add?
40. Retryer: wrap a function with maximum attempts, an interval, and exponential backoff (for example capped at 16 s). Rethrow the last error. Make it extensible. [Reported] [2][10][32]
    - How would you plug in jitter, retry-on-specific-exceptions or a time budget?
    - How do you unit-test timing without real sleeps?
41. Key store with sum dependencies: `set_value(k, v)`, `set_sum(k, [keys])` (keys may repeat) and `get_value(k)`. Updates must cascade to dependents. [Reported] [3][24]
    - What happens on a cycle?
    - Choose between eager propagation and lazy recompute for a workload with frequent writes and rare reads. [Reported] [2]
42. Design Excel Sum Formula (LC 631). [Reported] [4][5]
43. Banking class: deposit, withdraw, transfer, balance, and balance change over (start, end], where the boundary convention matters. [Reported] [2][3][34]
    - Your tests fail on a boundary. Which spec detail did you assume?
44. Refund allocation: given payments (method, date, amount) and existing refunds, allocate a new refund in full to one payment before the next. Prioritize by method

(CREDIT, then CREDIT_CARD, then PAYPAL), and within a method by most recent payment. [Reported] [2][3][42]
    - How do earlier partial refunds change the remaining refundable amount per payment?
    - How do you design the data structures when the interviewer gives only examples? [Reported] [42]
45. URL query parser: parse `?k=v&...` into a map. Duplicate keys become lists, a bare key means true, and percent-encoded separators (%26, %3D) are decoded *after* splitting. [Reported] [2][3][26]
    - In a typed language, how do you return values that may be a string or a list?
46. Ski tournament: in a DAG of checkpoints with point values and edge travel times, find the path from start to any end node that maximizes points minus time, and print the path. [Reported] [3][17][42]
    - Is the input guaranteed acyclic? What if it is not?
    - Is Dijkstra correct here, and why or why not?
47. Task scheduling with deadlines: unit-time tasks with a deadline and a reward. Return the order that maximizes total reward. [Reported] [4][28]
    - Why does the naive greedy fail, and what is your optimal-runtime approach? [Reported] [28]
    - How do you find the latest free slot before a deadline quickly? [Reported] [2]
48. Maximum Profit in Job Scheduling (LC 1235). Follow-up: output the chosen jobs. [Reported] [5][38]
49. Account registration deduplication: accounts share email, phone or government ID. Return all duplicates of a given account. [Reported] [2][3][4]
    - Generalize from two attributes to an arbitrary attribute set. [Reported] [2]
    - What changes if new accounts stream in continuously?
50. Build the smallest number from a list of digits (leading zeros dropped). Part 2: the smallest permutation that is >= a lower bound. [Reported] [2][30][55]
51. Intersection of two linked lists where either list may contain a cycle (four structural cases). [Reported] [2][3][5]
52. Multi-stream round-robin iterator: read bytes from several character streams in round-robin order with `has_next` and `next`. [Reported] [2][37]
53. Calculate board score: for every cell, add the length of the same-character run going right and going down. R and C go up to 2000. [Reported] [3][53]
54. Run-length encoding of an int array. Follow-ups: arithmetic-sequence RLE, and an iterator-based decoder for each encoding. [Reported] [25]
55. Sum tree: leaves hold numbers and parents hold the sum of their children. Return any node's value, then support mutations (leaf to subtree, subtree to leaf) while keeping reads correct. [Reported] [29]
56. Payment data store: `insert(ts, value, id)`, `get(id)` and `getValueInRange(start, end)`. [Reported] [33]
57. Keyword tagging in reviews: wrap keywords as `[tag]{keyword}` case-insensitively. Overlapping matches go to the longer keyword. [Reported] [3][10]
58. Replace text in a review using a replacement dictionary. [Reported] [3][10]
59. Shortest uncommon substring: for each string, find the shortest substring that appears in no other string (LC 3076). [Reported] [4][5][31]
60. Maximum Candies You Can Get from Boxes (LC 1298). It was given with no interface, so you define the API and write passing tests. [Reported] [5][14][36]

61. Office layout grid (-1 wall, 0 open): find the maximum reachable open area and count the unreachable rooms. [Reported] [36]
62. Two water jugs: return the minimum operations to measure a target volume **and** the sequence of states. [Reported] [10][27]
63. Given three numbers, list every way the first two combine with + - * / to equal the third. Follow-up: allow parentheses. [Reported] [45]
64. Card pile: by moving the top card to the bottom, can you reach N identical consecutive cards? [Reported] [44]
65. Parse strings of letter-score tokens and compute per-letter totals. [Reported] [43]
66. Find Median from Data Stream (LC 295). [Reported] [5][46]
67. Chain booking (maximum cascading reservations), infection spread time on a grid, maze score via shortest path, and highlight phrases in a review. [Reported] [3]
68. Compute browsing metrics in Python from event logs. [Reported] [3]
    - Which metrics would you compute per session versus per user, and how do you define a session?
69. Durable key-value store serialization, an in-memory database with SQL operations, and an in-memory object manager with TTL. [Reported] [2][3]
70. Car-speed controller: through a socket API (`getStatus`, `setStatus(acceleration)`), hold a simulated car at 29 mph. [Reported] [58]

C. LeetCode Airbnb-tag problems (company tag; frequency per the mirrors)

71. Smallest Sufficient Team (bitmask DP). It is the joint top problem in LeetCode's Airbnb tag for the last 3 months. [Reported] [4][5]
72. Shopping Offers (DP and backtracking with bundles, the LeetCode cousin of the menu problem). [Reported] [5][16]
73. Accounts Merge. [Reported] [4][5]
74. Smallest Common Region. [Reported] [5][6]
75. Detonate the Maximum Bombs. [Reported] [4][5]
76. Flatten 2D Vector, plus a remove variant. [Reported] [3][5]
77. Simple Bank System (LC 2043). [Reported] [5][6]
78. Trapping Rain Water. [Reported] [5][6]
79. Shortest Path to Get All Keys (LC 864). It was asked in the same round as Palindrome Pairs. [Reported] [5][39]
80. Minimum Number of Flips to Convert Binary Matrix to Zero Matrix. [Reported] [5][6]
81. Mini Parser and Basic Calculator II. [Reported] [5][6]
82. Contains Duplicate II and III. [Reported] [5][6]
83. Strings Differ by One Character. [Reported] [5][6]
84. Number of Ways to Build House of Cards. [Reported] [5][6]
85. Minimum Window Substring and Minimum Window Subsequence. [Reported] [5][6]
86. Making a Large Island. [Reported] [5][6]
87. All O`one Data Structure. [Reported] [5][6]
88. Minimum Cost to Reach Destination in Time. [Reported] [5][6]
89. Time Based Key-Value Store. [Reported] [6][12]
90. Coin Change and Combination Sum. [Reported] [5][17]
91. Easier tagged warm-ups: Two Sum, Valid Parentheses, Add Two Numbers, Add Strings, Merge k Sorted Lists, Happy Number (one candidate got three problems in

60 minutes), Maximal Square, Robot Bounded in Circle, Convert to Base -2, Nth Digit, Consecutive Numbers Sum, Can Place Flowers, Subarray Product Less Than K and Maximum Subarray. [Reported] [5][6][47]

D. ML/experimentation-flavored practical coding (role-specific, not yet reported)

None of these were reported for Airbnb. They follow the same practical, "make it run with tests" format and fit an MLE role on experimentation, search ranking or pricing.

92. Deterministic experiment bucketing: given user_id, an experiment salt and variant weights, return the assigned variant. [Likely]
    - How do you show that assignment is uniform and independent across two experiments?
    - How would you ramp from 5% to 50% without reassigning users already exposed?
93. From raw exposure and conversion logs (user, experiment, variant, ts, event), compute per-variant conversion, the absolute and relative lift, and a 95% confidence interval. [Likely]
    - How do you handle a user exposed to both variants, and events that happen before exposure?
94. Implement a sample-ratio-mismatch check for an expected 50/50 split and decide whether to alert. [Likely]
95. Implement CUPED or regression adjustment on a per-user metric with a pre-period covariate, and report the variance reduction. [Likely]
96. Implement a bootstrap confidence interval for the difference in a ratio metric (for example bookings per search). [Likely]
    - Why is resampling individual searches wrong when the unit of randomization is the user?
97. Implement NDCG@k for ranked search results with graded relevance (booking > click > impression). [Likely]
    - How do you handle queries with no relevant results?
98. Implement team-draft interleaving of two rankers' result lists and credit clicks to each ranker. [Likely]
99. Re-rank a scored list of listings to enforce host diversity: at most one listing per host in each window of k, as in the pagination problem. [Likely] [52]
100. Compute a rolling 7-day booking rate per listing from an event stream with bounded memory. [Likely]
101. Given availability calendars and nightly prices, compute the total price of a stay, including weekly-discount and cleaning-fee rules, and test the boundary nights. [Likely]
102. Reservoir-sample k events from a stream of unknown length, then make it weighted. [Likely]
103. Implement streaming AUC, or approximate quantiles, over a large prediction log without loading it all in memory. [Likely]
104. Write a sequential-testing guard: given daily cumulative results, decide whether a pre-registered stopping boundary has been crossed. [Likely]

E. Cross-cutting probes interviewers use in every coding round

105. Before you code: what are the input format, the output format and the tie-breaking rules? Is any of them ambiguous? [Reported] [16][21]
106. Write your own test cases now. Which edge cases did you choose, and why? [Reported] [1][2][35]
107. What are the time and space complexity, and where is the bottleneck for the largest stated input? [Reported] [1][56]
108. Your code fails one hidden test. How do you debug it in the remaining five minutes? [Likely] [17][34]
109. A new requirement arrives, such as returning the path instead of the cost, or k columns instead of one. How much of your code has to change? [Reported] [3][4][27]
110. If this ran in production behind an API endpoint, what would you validate, log or cache? [Likely] [18]

References

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview guide, mirrored on GitHub; based on 3 candidate reports, mainly 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb Software Engineer Interview Process" (TrueInterview guide, CC BY 4.0, mirrored; based on 41 candidate reports, 2025 Q4 to 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
3. "Airbnb" question list with report dates (algorithm, OOP and SQL questions, Sep 2025 to Sep 2026). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
4. "Airbnb" company guide (2025-2026 reports; merges LeetCode tag data, LeetCode Discuss, PracHub and Hello Interview). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
5. LeetCode company-tag mirror: Airbnb (All, Six Months and Three Months CSVs; updated June 2025). liquidslr/leetcode-company-wise-problems, GitHub, 2025. https://github.com/liquidslr/leetcode-company-wise-problems/tree/main/Airbnb
6. LeetCode company-tag mirror: airbnb/all.csv. snehasishroy/leetcode-companywise-interview-questions, GitHub. https://github.com/snehasishroy/leetcode-companywise-interview-questions/blob/master/airbnb/all.csv
7. "AirBnB Interview Questions" (31 classic problems with Java solutions). allaboutjst/airbnb, GitHub. https://github.com/allaboutjst/airbnb
8. "Airbnb_Interview" (Airbnb Beijing prep, 36 problems). hbsun2113, GitHub, 2019-2020. https://github.com/hbsun2113/Airbnb_Interview
9. "airbnb-interview-problems". shua2018ti, GitHub, 2018. https://github.com/shua2018ti/airbnb-interview-problems
10. "airbnb_interview_questions.md". warrenzhu25/system-design, GitHub. https://github.com/warrenzhu25/system-design/blob/master/airbnb_interview_questions.md
11. "Interview formats of top companies" (Airbnb section). Tech Interview Handbook (yangshun), GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/

apps/website/contents/interview-formats-top-companies.md
12. "Airbnb coding interview questions: the problems that actually show up...". yhzl350/airbnb-dp-interview, GitHub, 2026. Promotional page with unverified claims. https://github.com/yhzl350/airbnb-dp-interview
13. Airbnb 1point3acres interview-post mirror (folder index). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/tree/main/company%20wise/airbnb/1p3a
14. 1p3a post #40: project deep dive, coding LC 1298, ML design (LTV, host pricing). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
15. 1p3a post #79: two ML design rounds (listing recommendation, price prediction), coding ConnectN OOD, ML experience round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
16. 1p3a post #37: compilation of 16 algorithm questions with clarification tips (menu, Connect 4, select property; LC 336/864/284/1298/1557/1235). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/37.md.md
17. 1p3a post #72: phone screen Connect 4 with print, Combination Sum, onsite ski DAG problem. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/72.md
18. 1p3a post #34: split stay problem statement. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/34.md
19. 1p3a post #44: split stay; O(N^2) judged not good enough. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/44.md
20. 1p3a post #15: hotel split stay; "bug free, all cases must run". https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/15.md
21. 1p3a post #38: group-size property selection DP. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/38.md
22. 1p3a post #10: menu min cost with bitmask; DFS then DP follow-up. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/10.md
23. 1p3a post #53: rental services min-cost variant. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/53.md
24. 1p3a post #65: set_value / set_sum / get_value key store. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/65.md
25. 1p3a post #87: RLE with arithmetic-sequence and iterator follow-ups. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/87.md
26. 1p3a post #100: URL query parser with percent decoding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/100.md
27. 1p3a post #101: cheapest itinerary with max layovers returning the route; two-jug problem. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/101.md
28. 1p3a post #104: task deadlines and rewards; greedy rejected. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/104.md
29. 1p3a post #55: G10 full-stack; sum-tree with mutations; retry higher-order function. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/55.md
30. 1p3a post #33: smallest number from digits; lower-bound follow-up. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/33.md
31. 1p3a post #27: shortest uniquely-mapping substring. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/27.md
32. 1p3a post #69: Connect 4 phone screen (about 10 tests, column-height optimization); onsite retryer with increasing interval. https://github.com/FlowSight/coding/blob/mai

n/company%20wise/airbnb/1p3a/69.md

33. 1p3a post #64: payment data insert/get/getValueInRange. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/64.md
34. 1p3a post #98: banking system with timestamped balance changes. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/98.md
35. 1p3a post #12: menu bundles up to 3 with 5+ tests; terrain print and water drop. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/12.md
36. 1p3a post #24: LC 1298 with no interface given, tests required; office layout grid. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/24.md
37. 1p3a post #18: multi-stream iterator. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/18.md
38. 1p3a post #97: LC 1235 variant; running code on the spot. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/97.md
39. 1p3a post #50: coding round with LC 336 and LC 864. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/50.md
40. 1p3a post #85: simplified text formatting with column follow-up; task-score scheduling. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/85.md
41. 1p3a post #75: room allocation (set cover). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/75.md
42. 1p3a post #4: refund rules with examples; ski problem input format. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/4.md
43. 1p3a post #82: per-letter score aggregation. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/82.md
44. 1p3a post #90: card pile N-consecutive problem. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/90.md
45. 1p3a post #17: three-number arithmetic combinations with parentheses follow-up. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/17.md
46. 1p3a post #81: LC 295 in a 45-minute coding round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/81.md
47. 1p3a post #36: three LeetCode problems in 60 minutes (incl. Happy Number). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/36.md.md
48. 1p3a post #95: count of meal orders summing to budget. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/95.md
49. 1p3a post #68: menu exact-target combinations. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/68.md
50. "Fixed-Width Text Formatting (Multi-Column Follow-up)" problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/Fixed-Width%20Text%20Formatting%20(Multi-Column%20Follow-up)/qs.md
51. "Fill Layover Hours by Booking Experiences" problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/Fill%20Layover%20Hours%20by%20Booking%20Experiences/qs.md
52. "pagination" problem statement (host_id, listing_id, score, city). FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/pagination/qs.md
53. "Calculate Board Score" problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/Calculate%20Board%20Scor

e/qs.md

54. "guess word" (guess-number variants). FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/guess%20word/qs.md
55. "perm_qs 1" (smallest permutation >= lower bound). FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/perm_qs%201/qs.md
56. 1p3a post #56: menu DP, with the follow-up to optimize for the at-most-3-items constraint. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/56.md
57. "buddy list" follow-up statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/buddy%20list/qs.md
58. "car speed" (socket-based car speed controller) problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/car%20speed/qs.md

03. ML System Design: Search Ranking, Embeddings, Pricing, Trust, Recommendations and More

> Research note (read first). Live web research was not possible when this chapter was written. The shared web-search budget for the run was already used up. The egress proxy also refused connections (DNS failure or HTTP 403) to glassdoor.com, interviewquery.com, prepfully.com, arxiv.org and medium.com, so no page could be fetched or re-read. The chapter is therefore built from what was already known about Airbnb's published ML systems and its widely described interview loop.
>
> Tagging policy, applied strictly: **no question in this chapter is tagged [Reported]**, because no candidate report could be verified for this run. Every question is tagged **[Likely]**. That means it is an inference from the role, the senior level and the systems Airbnb has described publicly. A citation next to a [Likely] question points to the Airbnb paper or blog post that describes the underlying system. It does **not** mean that anyone was asked the question. References marked "URL not verified" give the title and the blog index rather than a guessed article URL. Before the onsite, cross-check this list against Glassdoor, Blind and your recruiter's prep material [13][14][15].

Format notes

- **Slot and length.** Airbnb's loop is widely described as including one ML/ML-system-design round for MLE candidates. It runs about 45-60 minutes and is held over video, with a shared doc or whiteboard [13][14]. Chapter 01 of this guide covers the full loop.
- **Interviewer.** Usually a senior or staff MLE from an ML-heavy area such as Search/Relevance, Pricing, Trust, Experiences/Recommendations or a platform team. Expect the interviewer to steer toward their own domain [Likely].
- **Shape of the round.** Typically one open-ended Airbnb product prompt ("design search ranking", "design a fraud model"). You are expected to drive it: clarify the business goal, pick the objective and labels, design data and features, choose a model, plan offline evaluation and online experimentation, cover serving and latency, and plan monitoring and iteration [Likely; 1][2].
- **Senior bar (L5).** Signals that the round assesses:
  - You frame the problem as a marketplace problem: guest and host, supply and demand.
  - You choose labels and objectives with care, including delayed or censored outcomes.

    - You propose a pragmatic baseline before deep models.
    - You reason about position bias, feedback loops and interference.
    - You design the **launch experiment** yourself. This matters especially for an experimentation-flavored role.
    - You discuss trade-offs without heavy prompting.

  Airbnb's own search papers emphasize simple baselines, failed ideas and lessons learned. Expect interviewers to reward the same humility and iteration [Likely; 1][2].

- **Experimentation weight.** For this role, assume every design will end with "how would you launch and measure it?" Expect probes on interleaving, guardrails, two-sided interference and long-horizon metrics [Likely; 8][9][17].

Questions

A. Search ranking for stays (core prompt)

1. Design the search ranking system for Airbnb stays: a guest enters a location, dates and guest count, and you return an ordered list of listings. [Likely] [1][2]
    - What is the business objective: bookings, booking value, guest satisfaction or long-term host and guest retention? How do you encode it?
    - What are the stages (candidate retrieval, filtering, first-pass ranker, second-pass re-ranker), and what is each stage's latency budget?
2. What label would you train on, given that bookings are rare and many searches end with no booking? [Likely] [1]
    - How would you use clicks, listing-page views, wishlist saves, contacts to host and bookings as weaker labels or auxiliary tasks?
    - How do you treat a booking request the host later rejected or cancelled?
3. How would you frame ranking: pointwise, pairwise or listwise? Why? [Likely] [1][2]
    - How would you build pairs from one search session with a single booked listing?
4. How do you correct for position bias in logged search data? [Likely] [2]
    - How would you model position as a feature during training, and what do you set it to at inference time?
    - How would you validate that your debiasing helped, rather than just shifting the distribution?
5. Which feature families would you build for listings, queries, guests and query-listing pairs? [Likely] [1]
    - How do you normalize price so that the model generalizes across markets and currencies?
    - How do you represent location relevance (distance to map center, the neighborhood the guest wanted)?
6. Your first model was a GBDT. When would you move to a neural network, and what would you expect to break? [Likely] [1]
    - How do you handle feature scaling, sparse categorical features and very wide value ranges in a neural ranker?
7. The ranker strongly prefers cheaper listings. Is that a bug or correct behavior? How would you investigate? [Likely] [1][2]

    - How would you make the model price-aware without hard-coding a monotonic price penalty?
8. How do you handle cold-start listings that have no booking or review history? [Likely] [2]
    - How do you estimate engagement for a new listing, and how do you give it controlled exposure without hurting conversion?
9. How would you add diversity to the results so the top of the page is not ten near-identical listings? [Likely] [5]
    - How would you measure diversity, and how would you trade it off against relevance in the loss?
10. How would you rank listings shown as map pins, where there is no clear top-to-bottom position? [Likely] [2]
    - How does user attention on a map differ from attention on a list, and how does that change your discounting?
11. How would you model the whole guest search journey (many searches across days before one booking) rather than a single search? [Likely] [6]
    - What would a multi-task model with heads for clicks, long views, wishlist and booking look like? How do you weight the heads?
12. How do you handle flexible-date or flexible-location searches ("I'm flexible", category browsing)? [Likely] [11]
13. How would you incorporate host-side outcomes (acceptance, cancellations, host earnings) into guest-facing ranking? [Likely] [1]
14. Offline NDCG went up but the online A/B test showed flat bookings. Walk me through your debugging plan. [Likely] [1][2]
    - How would you check for training-serving skew, feature leakage and a mismatch between offline and online candidate sets?
15. What offline evaluation set would you build, and how do you avoid leakage from future data or from repeat sessions by the same guest? [Likely] [1]

B. Candidate retrieval and serving infrastructure for search

16. Design candidate retrieval that cuts millions of listings to a few thousand within strict latency, respecting date availability, guest count and map bounds. [Likely] [12]
    - Where would embedding-based retrieval fit next to hard filters, and how do you combine them?
17. How would you train a two-tower model for embedding-based retrieval of listings? [Likely] [12]
    - What is your negative-sampling strategy (in-batch, same-market, hard negatives)?
    - How do you handle availability, which changes daily and cannot easily live inside the embedding?
18. How would you serve a neural ranker at p99 latency under 200 ms for thousands of candidates per query? [Likely]
    - Which features are precomputed offline, which are computed in real time, and how do you keep them consistent?
19. How do you prevent training-serving skew in features such as "listing price for these dates" or "number of recent bookings"? [Likely] [1]

20. How often would you retrain the ranking model, and how would you detect that it has gone stale? [Likely]
21. What would you log at serving time so you can retrain, debias and debug later? [Likely] [2]

C. Listing embeddings and real-time personalization

22. Design listing embeddings trained from guest click sessions, and use them for similar-listing recommendations and in-session personalization. [Likely] [3][10]
    - How do you define a session and its context window?
    - What negatives would you use? Why might random negatives make embeddings that only learn geography?
23. How would you change the embedding objective so that the booked listing influences every listing in the session? [Likely] [3]
24. How would you build user-type and listing-type embeddings to capture long-term preferences from sparse booking histories? [Likely] [3]
25. How would you create an embedding for a brand-new listing with no sessions? [Likely] [3]
26. How would you evaluate embeddings offline before using them in production? [Likely] [3]
    - What would a "rank of the eventually booked listing among the clicked listings" evaluation look like?
27. How would you turn in-session clicks and skips into real-time ranking features? [Likely] [3]
    - What streaming infrastructure is needed, and what is the freshness requirement?
28. When would you replace skip-gram embeddings with a sequence model or transformer over user actions? What would you expect to gain? [Likely]

D. Pricing (host price suggestions / Smart Pricing)

29. Design a system that suggests a nightly price to hosts for each future date. [Likely] [4][16]
    - What is the objective: booking probability, expected host revenue, or marketplace occupancy?
    - Why can't you train directly on "the optimal price" as a label?
30. How would you predict booking probability as a function of price, lead time and listing quality? [Likely] [4]
    - How do you handle price being confounded with demand, because hosts raise prices when they expect demand?
31. How would you estimate price elasticity per listing or per market from observational data? [Likely] [4]
    - When would you need randomized price experiments, and what ethical and host-trust constraints apply?
32. Hosts can accept, modify or ignore your suggestion. How do you evaluate the model offline and online? [Likely] [4]
    - What is your unit of randomization, and how do you handle spillover between treated and control hosts in the same market?

33. How would you forecast market demand for a city on a given date (events, holidays, seasonality)? [Likely] [16]
34. How would you prevent the pricing model from producing extreme or volatile suggestions that destroy host trust? [Likely]
35. How would you design total-price display or fee-related ranking changes, and how would you measure whether guests trust the price more? [Likely]
36. A pricing-model launch raised bookings but cut average host revenue. How do you decide? [Likely] [17]

E. Fraud, risk and trust and safety

37. Design a model that scores every booking for payment-fraud risk in real time. [Likely] [18]
    - Chargeback labels arrive weeks later. How do you train and evaluate with delayed labels?
    - How do you handle label censoring, where blocked transactions never reveal whether they were fraud?
38. Rather than block or allow, how would you design graduated friction (ID verification, extra payment checks) driven by risk scores? [Likely] [18]
    - How do you choose thresholds per action, given the cost of false positives (lost good guests) and false negatives?
39. Design an account-takeover detection system for guest and host logins. [Likely] [18]
40. Design a system that predicts and prevents unauthorized parties at listings. [Likely] [19]
    - What features are legitimate and fair to use, and which risk proxy discrimination (for example by age)?
    - How do you measure the system's impact when the outcome (a party happened) is rarely observed?
41. Design a fake or scam listing detector at listing-creation time. [Likely] [19]
    - How do you combine text, image, host-history and graph signals (shared devices, payment instruments)?
42. How would you use graph-based features or graph neural networks to detect fraud rings? [Likely]
43. Fraudsters adapt to your model within weeks. How do you design for adversarial drift? [Likely]
    - How do you keep a clean holdout to measure true fraud rates when the model blocks most of it?
44. How do you A/B test a fraud model when exposing the control group to fraud has real costs? [Likely] [18]
45. How do you build fairness checks into a trust model that affects who can book? [Likely] [19]

F. Recommendations (Experiences, similar listings, discovery surfaces)

46. Design search ranking for Airbnb Experiences, which has far less data than stays. [Likely] [7]
    - How would you stage the system as data grows (heuristics, then GBDT, then personalization, then online scoring)?

47. Design "similar listings" recommendations on the listing detail page. [Likely] [3]
    - What is the right objective: clicks, bookings, or rescuing guests whose first choice is unavailable?
48. Design homepage recommendations for a logged-in guest with no active search ("where should I go next?"). [Likely]
49. Design personalized marketing or email recommendations (destinations, wishlisted listings with price drops). [Likely]
    - How do you measure incremental lift rather than attribution from people who would have booked anyway?
50. Design recommendations for cross-selling Experiences or Services to guests with an upcoming stay. [Likely]
51. How would you recommend wishlist listings or destinations for group trips where several people collaborate? [Likely]
52. How do you avoid a popularity feedback loop in which top listings get more exposure, more bookings and more exposure? [Likely] [5]

G. Host acceptance, response time and other time-to-event predictions

53. Design a model that predicts the probability a host will accept a booking request, and use it in ranking. [Likely] [1]
    - How do you avoid unfairly penalizing a host for a single rejection?
    - How do you use host preferences (trip length, lead time, gaps between bookings) as features?
54. Design a model that predicts host response time to a guest message or booking request. [Likely]
    - How do you handle censoring for messages that never get a reply?
55. Predict the probability that a confirmed booking will be cancelled by the host or by the guest. [Likely]
    - How would you use this in ranking, in host coaching, or in guest-protection policy?
56. Predict booking lead time and length of stay for a search, and explain how they would change ranking. [Likely] [1]
57. Design a model that estimates customer-support wait time or resolution time, and routes cases to the right agent queue. [Likely] [20]
58. Which model families would you consider for time-to-event problems (survival models, hazard models, classification at fixed horizons)? Why? [Likely]

H. Categories, content understanding and listing quality

59. Design the system behind Airbnb Categories (for example "Beachfront", "Treehouses"), which assigns listings to themed browse categories. [Likely] [11]
    - How do you combine ML candidates with human-in-the-loop review, and how do you budget human review?
    - How do you rank listings within a category, and how do you rank the categories themselves for a user?
60. Design an image-understanding pipeline that detects amenities and room types from listing photos. [Likely] [21]

    - How would you get labels cheaply, and how do you measure precision when amenities drive search filters?
61. How would you choose and order a listing's cover photo to maximize engagement without misleading guests? [Likely]
62. Design a listing-quality score used in ranking and host coaching (photos, description, completeness, reviews). [Likely]
    - How do you keep the score from simply rewarding listings that are already popular?
63. Design a system that detects inaccurate or misleading listing descriptions by comparing text, photos and reviews. [Likely]
64. How would you use LLMs or multimodal models for listing understanding at Airbnb's scale while controlling cost? [Likely]
65. How would you translate or localize listing content and decide when machine translation is good enough to show? [Likely]

I. Reviews: quality, ranking and trust

66. Design a system that ranks reviews on a listing page so guests see the most helpful reviews first. [Likely]
    - How do you define "helpful" without explicit helpfulness votes?
67. Design a fake or retaliatory review detector. [Likely] [19]
68. Design a system that extracts aspects (cleanliness, location, accuracy) from review text and feeds them into search or listing quality. [Likely]
69. Design LLM-generated review summaries or highlights for a listing. [Likely]
    - How do you evaluate faithfulness, so the summary claims nothing the reviews do not support?
    - How do you A/B test whether summaries change booking or cancellation behavior?
70. Ratings are highly skewed toward 5 stars. How do you build a reliable quality signal from them? [Likely]
    - How would you apply Bayesian shrinkage for listings with few reviews?
71. How would you predict a guest's post-stay satisfaction before the stay, and how would you use it in ranking? [Likely] [1]

J. Messaging, customer support and LLM systems

72. Design an ML system that classifies the intent of guest-host messages to route or auto-suggest replies. [Likely] [20]
73. Design an LLM-based customer-support assistant that resolves common issues (cancellations, refunds) and escalates the rest. [Likely] [20]
    - How do you evaluate it offline, monitor hallucinations and policy errors, and measure deflection versus satisfaction online?
    - What guardrails are needed for actions involving money or safety?
74. Design suggested replies for hosts. How do you avoid generating replies that commit the host to things they did not intend? [Likely]
75. How would you build retrieval-augmented generation over help-center articles and booking context? [Likely]

76. How would you decide whether to fine-tune, prompt or distill a model for a high-volume support classifier? [Likely]

K. Experimentation and measurement inside the ML design round

77. For your ranking model, what is the primary metric of the launch A/B test, and what are the guardrails? [Likely] [8][17]
    - Why might "bookings" be a better primary metric than "booking requests" or "clicks"?
78. What randomization unit would you use for a search ranking test (guest, session, search, listing, market)? Why? [Likely] [8]
79. Explain why a guest-randomized test of a ranking change can be biased in a two-sided marketplace with limited inventory. [Likely] [8]
    - How could market-level, cluster or switchback designs help? What do they cost in statistical power?
80. When would you use interleaving instead of a classic A/B test for ranking? [Likely] [9]
    - How do you attribute a booking to one ranker in an interleaved result list?
    - How do you validate that interleaving and A/B testing agree on direction?
81. Bookings happen days after search, and stays happen weeks later. How do you get a sensitive, timely decision metric? [Likely] [8][17]
    - How would you build and validate a surrogate or predicted long-term value metric?
82. How would you reduce variance in a booking-conversion experiment (CUPED, stratification, trimming)? [Likely] [8]
83. How do you detect and handle sample ratio mismatch, bot traffic or logging bugs in a ranking experiment? [Likely] [8]
84. How do you handle novelty and primacy effects, and seasonality around holidays, in experiment readouts? [Likely]
85. Your model wins overall but loses badly in one market or one guest segment. What do you do? [Likely]
86. How would you set up a long-term holdout to measure the cumulative impact of many ranking launches? [Likely]
87. How would you measure the impact of a pricing or fraud model where the treatment changes host or fraudster behavior over time? [Likely] [4][18]
88. How would you use counterfactual or off-policy evaluation from logged ranking data before running an online test? [Likely] [2]
    - What propensities do you need to log, and how much randomization would you inject?
89. How do you trade off guest-side and host-side metrics in a single ship decision? [Likely] [17]

L. Monitoring, reliability and iteration (senior depth)

90. What would you monitor once your ranking or fraud model is live (data drift, feature nulls, score distributions, business KPIs)? [Likely]
    - Which alerts would page an on-call engineer, and which go to a dashboard?
91. How would you roll out a new model safely (shadow scoring, canary, ramp, automatic rollback)? [Likely]

92. A feature pipeline broke silently and the model has served on stale features for three days. How do you detect it, mitigate it and prevent it from happening again? [Likely]
93. How do you decide what to build next after launch: more features, a bigger model, or better labels? [Likely] [1][2]
94. How would you reduce serving cost by 50% without measurable loss in bookings? [Likely]
    - What is your approach to distillation, caching and candidate-set pruning?
95. How would you design the system so that other teams (Experiences, Services, Long-term stays) can reuse your ranking stack? [Likely]
96. How would you explain to a product manager why a model change that "looks worse" in qualitative review should still ship? [Likely]
97. Which parts of this design would you not use ML for at all, and why? [Likely] [1]
98. What are the main fairness and discrimination risks in Airbnb ranking and trust systems, and how would you audit for them? [Likely] [19]

M. Variations and curveball prompts

99. Design ranking for long-term stays (28+ nights). What changes from nightly stays? [Likely]
100. Design ranking for Airbnb Services or new verticals with almost no historical data. [Likely] [7]
101. Design autocomplete and query understanding for the search box (locations, landmarks, categories). [Likely]
102. Design a model that predicts which markets will face supply shortages for an upcoming event, and recommend actions to recruit hosts. [Likely]
103. Design a host-onboarding recommendation system that tells new hosts what to improve first. [Likely]
104. Design a system that detects duplicate listings (the same property listed several times or across accounts). [Likely]
105. Design a "price drop" or "rare find" badge. How would you verify that badges are accurate and actually help guests? [Likely]

References

None of the sources below could be opened for this chapter: the egress proxy blocked arxiv.org, medium.com, glassdoor.com and the others. Each is listed from prior knowledge. Entries marked "URL not verified" give the title and the Airbnb tech blog index rather than a guessed article URL. Search the title there. Confirm every link before relying on it.

1. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD 2019. https://arxiv.org/abs/1810.09591
2. Haldar et al., "Improving Deep Learning for Airbnb Search", arXiv / KDD 2020. https://arxiv.org/abs/2002.05515
3. Grbovic & Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
4. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219830

5. Abdool, Haldar et al., "Learning To Rank Diversely At Airbnb", arXiv, 2022 (CIKM 2023). https://arxiv.org/abs/2210.07774
6. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning", arXiv / KDD 2023. https://arxiv.org/abs/2305.18431
7. Grbovic, "Machine Learning-Powered Search Ranking of Airbnb Experiences", Airbnb Tech Blog (Medium), 2019. https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789
8. "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
9. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Tech Blog (Medium), 2022. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
10. Grbovic, "Listing Embeddings in Search Ranking", Airbnb Tech Blog (Medium), 2018. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
11. "Building Airbnb Categories with ML and Human-in-the-Loop", Airbnb Tech Blog, 2022-2023. URL not verified. Search via https://medium.com/airbnb-engineering
12. "Embedding-Based Retrieval for Airbnb Search", Airbnb Tech Blog, 2025. URL not verified. Search via https://medium.com/airbnb-engineering
13. "Airbnb Machine Learning Engineer Interview Questions", Glassdoor (candidate reports), ongoing. https://www.glassdoor.com/Interview/Airbnb-Machine-Learning-Engineer-Interview-Questions-EI_IE391850.0,6_KO7,32.htm
14. "Airbnb Interview Process & Questions", IGotAnOffer, 2023-2024. https://www.igotanoffer.com/blogs/tech/airbnb-interview-process
15. "Airbnb Interview Discussions", LeetCode Discuss (search "Airbnb"), ongoing. https://leetcode.com/discuss/interview-question?currentPage=1&orderBy=hot&query=airbnb
16. "Learning Market Dynamics for Optimal Pricing", Airbnb Tech Blog, 2018. URL not verified. Search via https://medium.com/airbnb-engineering
17. "How Airbnb Measures Future Value to Standardize Tradeoffs", Airbnb Tech Blog, 2022. URL not verified. Search via https://medium.com/airbnb-engineering
18. "Fighting Financial Fraud with Targeted Friction", Airbnb Tech Blog, 2018. URL not verified. Search via https://medium.com/airbnb-engineering
19. Airbnb Trust & Safety and anti-discrimination posts (party prevention, Project Lighthouse), Airbnb Tech Blog and Airbnb Newsroom, 2020-2024. URL not verified. Search via https://medium.com/airbnb-engineering
20. Airbnb posts on message-intent classification and customer-support ML / LLM systems, Airbnb Tech Blog, 2019-2025. URL not verified. Search via https://medium.com/airbnb-engineering
21. Airbnb posts on amenity detection and room-type classification from listing photos, Airbnb Tech Blog, 2018-2019. URL not verified. Search via https://medium.com/airbnb-engineering

04. A/B Testing and Experimentation Deep Dive: ERF, Marketplace Interference, Variance Reduction, Interleaving and Sequential Testing

> Research note (read first). No live research was possible for this chapter. The shared web-search budget for this run was already used up (0 of the planned 30-45 searches ran), and every WebFetch attempt failed DNS resolution (medium.com, igotanoffer.com). The questions below are therefore built from prior knowledge of Airbnb's published experimentation work and of its widely reported loop structure (see Chapter 01). **Every question is tagged [Likely].** None is tagged [Reported], because no candidate report could be opened and checked during this run. Citations such as "[Likely; 3]" point to the Airbnb publication that makes the topic a plausible probe. They do not mean the question was asked. Confirm each reference before relying on it. To fill the gap, rerun this chapter's research with a fresh search budget and look for real candidate reports on Glassdoor, Blind, 1point3acres and LeetCode Discuss.

Format notes

- **Where this material shows up.** Public descriptions of Airbnb's engineering loop (coding x2, system design, ML design, past-project deep dive, core values x2) do not list a separate "A/B testing" round for MLEs [9][10]. For an MLE role that names experimentation or A/B testing, expect it in three places: (a) the evaluation and launch half of the ML design round; (b) the past-project deep dive, with questions like "how did you know it worked?"; and (c) possibly a dedicated experimentation or analytics conversation with a data scientist or experimentation-platform engineer. Ask your recruiter which of these your loop includes. [Likely]
- **Duration and interviewer.** Usually 45-60 minutes with a senior or staff MLE, a data scientist (Airbnb's DS org has an Inference/Experimentation track) or an engineer from the experimentation platform team (ERF) [1][3]. [Likely]
- **The senior (L5) bar.** It goes beyond knowing the t-test. Expect to show you can:
    - spot when the standard A/B setup is biased (interference, novelty, selection, peeking);
    - choose the randomization unit and the metrics, and defend that choice;
    - quantify trade-offs between bias and variance and between speed and rigor;
    - turn ambiguous or conflicting results into a launch decision that Product and leadership accept.

Airbnb's own writing stresses that experiments run on a two-sided marketplace with long, rare conversion funnels (search, then contact or book, then stay, then review). Expect probes on exactly that [1][2][4][5]. [Likely]

- **Format of the questions.** Expect open-ended scenarios ("we changed X in search ranking; design the test; results came back like this; what now?"). Some rounds may add light whiteboard math, such as sample size, standard errors or CUPED adjustment. Expect few closed-form trivia questions. [Likely]

Questions

A. Airbnb experimentation platform (ERF) and experiment fundamentals

1. Walk me through how you would design an online A/B test for a new search-ranking model at Airbnb, from hypothesis to launch decision. [Likely; 1][2]
    - What is your primary metric, and why not just click-through rate?
    - What is the randomization unit, and what are you assuming by choosing it?
    - How long would you run it, and what determines that?
2. Airbnb's experimentation platform (ERF, the Experiment Reporting Framework) computes metrics for hundreds of concurrent experiments. If you were designing such a platform, what are its core components? [Likely; 3]
    - How would you implement deterministic, salted hashing for assignment so that concurrent experiments are independent?
    - How do you log exposures versus assignments, and why does the difference matter?
    - How would you make metric definitions consistent across teams (for example, through a central metrics layer)? [Likely; 11]
3. What is the difference between assignment and exposure (trigger) analysis, and when is an intent-to-treat analysis the right choice? [Likely]
    - The treatment only changes the page when a listing has more than 10 photos. Who should be in your analysis population?
    - How does triggering affect your power, and how do you translate a triggered effect back to a site-wide impact?
4. Many Airbnb users browse logged out and on several devices before booking. How does this affect randomization and metric attribution? [Likely; 1]
    - Would you randomize by cookie, device, user ID or something else? What breaks in each case?
    - How would you handle a user who is assigned on a logged-out device and then logs in?
5. What is a sample ratio mismatch (SRM)? How would you detect it, and what are its common causes on a web/mobile booking site? [Likely]
    - The treatment loads more slowly, so fewer treatment users fire the exposure log. What does that do to your results?
    - Would you ever analyze an experiment that fails the SRM check?
6. How would you run an A/A test, and what would you use it for on a mature platform? [Likely]

    - Your A/A tests show 9% significant results at alpha = 0.05 on one metric. What do you investigate?
7. Explain the p-value and the confidence interval of a difference in means to a product manager who wants a yes/no answer. [Likely]
8. How do you compute the sample size and minimum detectable effect (MDE) for a booking-conversion experiment? [Likely; 1]
    - Booking conversion is low and bookings are lumpy. How does that change the calculation?
    - The PM wants a two-week answer, but power analysis says six weeks. What options do you propose?
9. Your metric is a ratio, such as bookings per search or revenue per booking, where the analysis unit differs from the randomization unit. How do you compute its variance correctly? [Likely]
    - When would you use the delta method, and when a bootstrap?
    - What goes wrong if you treat each search as independent?
10. How do you handle heavy-tailed metrics like gross booking value, where a few long or luxury stays dominate? [Likely]
    - Would you winsorize/cap, log-transform or use a different estimator? What does each do to interpretation?
11. Airbnb's 2014 post describes an experiment whose result flipped depending on when it was checked, and a case where a browser-specific bug drove the result. What general lessons do you take from such cases? [Likely; 1]
    - How would you build a platform guard against "result depends on the day you looked"?
    - How would you segment by platform/browser without going on a fishing expedition?
12. How long should a booking experiment run given weekly seasonality and a lead time between search and check-in of days to months? [Likely; 1]
    - What do you lose if you stop after full weeks but before guests actually stay?
13. How would you design guardrails that block a launch automatically? What would the guardrail metrics be for a search change at Airbnb? [Likely; 4]
    - How do you set the guardrail threshold so it is neither too noisy nor too lax?
    - Who should be able to override a guardrail, and how should overrides be logged?

B. Metric choice: north stars, proxies and long-term value

14. What should the primary success metric be for a search-ranking experiment at Airbnb: bookings, booking value, nights booked, uncancelled bookings or something else? Defend your choice. [Likely; 2][5]
    - Why might "uncancelled bookings" or "booking with no host cancellation" be better than raw bookings?
    - How would you weigh guest-side gains against host-side outcomes?
15. Bookings are slow and rare, so the team wants a faster proxy metric (listing clicks, wishlist saves, contacts). How do you validate that a proxy is aligned with the north star? [Likely; 2]
    - Show me a scenario where the proxy improves and bookings drop.

    - How would you use historical experiments to measure proxy-to-north-star correlation, and what are the pitfalls?
16. Airbnb has written about putting a monetary future value on actions so that teams can compare trade-offs. How would you use such a metric in an experiment, and how could it mislead you? [Likely; 6]
    - How do you validate a model-based "future value" metric before letting it decide launches?
17. How would you measure long-term effects such as repeat booking, host churn or review quality that appear months after a test ends? [Likely]
    - What are the trade-offs of long-running holdouts?
    - How would you use surrogate indices or a long-term holdback group?
18. Design an overall evaluation criterion (OEC) for a pricing-suggestion feature shown to hosts. [Likely; 7]
    - What are the host-side metrics, the guest-side metrics and the marketplace-level metrics?
    - Host earnings rise but occupancy falls. Is that a win?
19. How do you choose between a mean metric (average bookings per user) and a proportion metric (percentage of users who book)? [Likely]
20. You track 50 metrics, and three moved significantly in the wrong direction. How do you decide whether to worry? [Likely]
    - How do you control for multiple comparisons without destroying power?
    - Which metrics would you pre-register as primary, secondary and guardrail?
21. How would you measure the quality of search results beyond conversion, for example relevance, diversity or "guest found what they wanted"? [Likely; 8]
    - How would you test a diversity objective whose value is not captured by short-term bookings?
22. Cancellation and refund outcomes happen after the experiment window closes. How do you build a metric that accounts for them without waiting months? [Likely]

C. Marketplace interference and two-sided experiments

23. Why can a standard user-randomized A/B test give a biased estimate in a two-sided marketplace like Airbnb? Explain it with an example. [Likely; 12][13]
    - In which direction is the bias usually, for a ranking change that helps treatment guests book popular listings?
    - When is the interference negligible?
24. A treatment ranking model steers guests toward high-quality listings. Treatment and control guests compete for the same limited inventory on the same dates. What does the A/B test overstate, and why? [Likely; 12]
    - How would you estimate the size of this cannibalization bias?
25. Compare guest-side (demand) randomization with listing/host-side (supply) randomization. When would you use each? [Likely; 13]
    - Which side do you randomize for a host pricing tool, and what interference does that leave?
    - What is the interference pattern when you randomize listings for a ranking boost?
26. Explain two-sided randomization (randomizing both guests and listings, then analyzing the cells). What bias does it reduce, and at what cost? [Likely; 13]

27. Airbnb collaborated on research about reducing interference bias in marketplace pricing experiments. How would you design a pricing experiment that limits interference? [Likely; 12]
    - How would cluster randomization by market or by "similar listings" help, and what does it do to variance?
    - How would you build listing clusters (for example, by geography and listing similarity) so that most competition stays within a cluster?
28. Compare switchback (time-based) designs, geo/market-level randomization and cluster randomization for an Airbnb search or pricing change. [Likely]
    - Why are switchbacks awkward when there is a long lead time between booking and stay?
    - With only a few hundred markets, how do you get enough power from a market-level experiment?
29. How would you detect interference empirically, for example by running the same treatment at two allocation levels (10% versus 50%)? [Likely; 13]
    - What would you conclude if the effect at 50% allocation is half the effect at 10%?
30. Market conditions matter. When supply is tight (peak dates), interference bias grows. How would you account for supply/demand balance in the analysis? [Likely; 13]
    - Would you report the effect separately for supply-constrained and demand-constrained markets?
31. A change makes guests book earlier. Total bookings over the quarter may not change, but the A/B test shows a lift. How would you detect this kind of intertemporal substitution (pull-forward)? [Likely]
32. How would you experiment on a host-side product, such as a new listing-quality nudge, when the outcome you care about (bookings) is driven by guests who are not in your experiment? [Likely]
33. Network effects among guests are weak on Airbnb, but interference through inventory is strong. Contrast this with social networks, and explain how the remedies differ. [Likely]
34. How would you use a market-level holdout ("launch everywhere except these N cities") to validate the global effect of a ranking system after launch? [Likely]
    - How do you choose holdout markets, and how do you analyze them (difference-in-differences, synthetic control)?
35. When randomization at the right level is impossible, how would you estimate the causal effect of a marketplace-wide change? [Likely; 14]
    - How does a counterfactual model (predict what would have happened without the change) work, and how do you validate it?
    - What assumptions does synthetic control rely on, and how would you check them?

D. Variance reduction: CUPED, stratification and regression adjustment

36. Explain CUPED. What is it, why does it reduce variance, and why does it not bias the treatment effect estimate? [Likely; 15]
    - How do you choose the covariate, and what is the optimal theta?
    - Why must the covariate be measured before the experiment starts?

37. Booking is rare, and many users have no pre-period activity (new or logged-out users). How does CUPED perform for them, and what would you do instead? [Likely]
    - Could you use a model-predicted covariate, for example a predicted booking probability (CUPAC-style)? What are the risks?
38. How do CUPED and regression adjustment with several covariates relate? When would you use one over the other? [Likely]
39. Explain stratified sampling versus post-stratification for variance reduction. Which strata would you use for Airbnb (market, platform, new versus returning guests)? [Likely]
40. You applied CUPED and the variance fell by only 3%. Why might that happen for a booking metric, and what else would you try? [Likely]
41. How does variance reduction change your sample-size calculation and your experiment duration? [Likely]
42. Can you combine CUPED with ratio metrics and with the delta method? What do you need to be careful about? [Likely]
43. When would you choose a more sensitive design over variance reduction in the analysis? Examples include interleaving, paired designs and within-user designs. [Likely; 8]

E. Interleaving for search ranking

44. What is interleaving, and why is it much more sensitive than an A/B test for comparing rankers? [Likely; 8]
    - Compare team-draft interleaving with balanced interleaving.
    - How do you assign credit for a booking, as opposed to a click, in an interleaved result list?
45. Airbnb has described using interleaving to speed up search-ranking experiments. How would you set it up so that the interleaving verdict agrees with A/B results? [Likely; 8]
    - How would you measure agreement between interleaving and A/B tests over a set of historical launches?
    - What would you do if interleaving says ranker B wins but the A/B test is flat?
46. Airbnb search is not a single click-and-done session. Guests search many times over days, and listings appear on map views and across pages. What complications does this create for interleaving? [Likely; 8]
    - How would you handle the same listing appearing in both rankers' lists, and duplicates across searches?
    - How do you attribute a booking that happens days later?
47. What can interleaving not tell you? Give examples of ranking changes that must still be A/B tested (for example, changes to price display, diversity or result-set size). [Likely; 8]
48. Design a two-stage experimentation funnel: offline evaluation, then interleaving, then A/B test. What are the gates between stages? [Likely; 2][8]
    - Offline NDCG improves but interleaving is flat. What is your hypothesis?

F. Offline-online gap and ML-specific experimentation

49. Your new ranking model improves offline NDCG by 2%, but the A/B test shows no booking lift. What are the possible reasons, and how do you debug? [Likely; 2][5]

    - Could position bias or feedback loops in the training data explain it?
    - How would you tell a modeling problem apart from an experiment-sensitivity problem?
50. Airbnb's deep-learning search papers describe many launches that improved offline metrics but failed online. How would you build an offline metric that better predicts online wins? [Likely; 2][5]
51. When training data come from the control model's own rankings, the treatment model is evaluated on data biased toward control. How does this affect A/B results, and how would you mitigate it? [Likely]
    - What role do exploration traffic or randomized-result buckets play?
52. How would you use off-policy evaluation (counterfactual estimators such as inverse propensity scoring) to pre-screen ranking models before an online test? What does it require from logging? [Likely]
53. Treatment and control models share a training pipeline. After launch, the new model's data feeds back into retraining. How would you measure that feedback-loop effect? [Likely]
54. How would you A/B test a model whose outputs are cached or precomputed, such as listing embeddings or nightly batch scores? [Likely; 16]
    - What contamination can occur if the treatment's embeddings are also used by control's similar-listings carousel?
55. How would you run an experiment on an ML model that serves several surfaces (search, similar listings, email recommendations) at once? [Likely]
56. How would you test a latency-increasing model (larger neural net) when latency itself hurts conversion? [Likely]
    - How would you separate the model-quality effect from the latency effect?
57. How would you experiment on a fraud or trust model, where false positives block real guests and the true outcome (fraud, chargebacks) arrives weeks later? [Likely; 17]
    - Is holding out a random set of transactions from the fraud model ethical or practical? What would you propose?

G. Sequential testing, peeking and decision rules

58. What goes wrong when a team peeks at an A/B test every day and stops as soon as $p < 0.05$? Quantify it roughly. [Likely; 1]
59. Explain sequential testing approaches (group sequential with alpha spending, always-valid p-values or mSPRT). Which would you build into an experimentation platform, and why? [Likely; 18]
    - What do you give up in power compared with a fixed-horizon test?
    - How would you explain an "always-valid" confidence interval on a dashboard?
60. Airbnb's 2014 post proposed a dynamic p-value threshold that is stricter early on. How does that compare with modern sequential methods? [Likely; 1]
61. When is early stopping for harm appropriate, versus early stopping for benefit? [Likely]
62. A test has run for its planned duration, and the p-value is 0.07. The PM wants two more weeks. What do you advise? [Likely]
63. How would you apply a Bayesian A/B testing approach to booking conversion? When does it help decision-making, and when does it mislead? [Likely]
    - How would you set priors from Airbnb's history of past experiments?

64. Explain the winner's curse in experimentation. Airbnb has written about correcting for selection bias when summing the impact of launched winners. How would you correct it? [Likely; 19]
    - Leadership adds up the measured lifts of 40 launched experiments, and the total far exceeds the actual business growth. Why?
65. How would you control the false discovery rate across hundreds of experiments and metrics on a platform? [Likely]

H. Novelty, primacy, learning and time effects

66. How do you detect novelty or primacy effects in an Airbnb experiment, for example a redesigned listing page? [Likely]
    - How would you plot and test the treatment effect over time or by days since first exposure?
    - Why are novelty effects harder to see when most guests visit Airbnb only a few times a year?
67. Guests take days to weeks to book. How would you distinguish a real long-term effect from a novelty spike in such a sparse-visit product? [Likely]
68. How do hosts learn and adapt to a new host-side feature, such as changing prices after a pricing tip? How would you design the experiment to capture the post-adaptation equilibrium? [Likely; 7]
69. How would you handle seasonality and holidays, for example a test that spans the summer peak or the New Year period? [Likely]
    - Would you pause, extend or reweight?
70. Experiments run during an external shock (for example a pandemic-scale demand drop or a major event in a city). How do you decide if the results still generalize? [Likely]
71. How would you use a long-term holdback after launch to measure decay in a feature's effect? [Likely]

I. Heterogeneous effects, segments and the launch decision

72. The overall effect is flat, but new guests show +3% and returning guests show -2%. How do you decide whether to launch, and whether to personalize? [Likely]
    - How do you avoid Simpson's paradox in this reading?
    - How do you know the segment effect is not a multiple-testing artifact?
73. How would you estimate heterogeneous treatment effects with ML (for example causal forests or uplift models), and how would you validate them? [Likely]
74. A treatment helps in large markets and hurts in small markets. What could explain this in a marketplace, and what would you do? [Likely]
75. The experiment improves bookings but reduces revenue per booking because guests choose cheaper stays. How do you make the launch call? [Likely; 6]
76. Results are significant on iOS and web but not on Android. How do you reason about it? [Likely; 1]
77. Walk me through the launch-review discussion you would lead when the primary metric is up, one guardrail is slightly down and the result is borderline significant. [Likely; 4]
    - What would you write in the experiment review document?

    - How do you communicate uncertainty to a VP who wants a single number?
78. How would you size the actual business impact of a launch, as opposed to the experiment's measured lift? Consider trigger rate, ramp-up, interference and decay. [Likely; 19]

J. Experiment infrastructure and ML-platform design for experiments

79. Design the assignment service for an experimentation platform that serves search, mobile apps and email, with consistent bucketing across surfaces. [Likely; 3]
    - How do layers or mutually exclusive groups work, and when do you need them?
    - How do you handle ramping from 1% to 50% without reshuffling users?
80. Design the metrics computation pipeline behind a system like ERF: daily aggregates for thousands of metrics across hundreds of experiments. [Likely; 3][11]
    - How do you make it incremental, backfill-able and consistent with the company's metric definitions?
    - How do you make CUPED and the delta method efficient at that scale?
81. How do you detect and handle interactions between concurrent experiments, for example two ranking experiments that both touch the same feature? [Likely]
82. How would you design an experiment-quality monitoring system that automatically checks SRM, logging gaps, pre-period imbalance and metric anomalies? [Likely; 4]
83. How would you architect shadow and canary deployment for a new ranking model before it enters an A/B test? [Likely]
    - What would you compare in shadow mode, and what kinds of failure would make you abort?
84. How would you store and expose historical experiment results so that teams can run meta-analyses, such as proxy validation or calibrating priors? [Likely]
85. How would you build self-serve power analysis into the platform, given that metric variances differ by market and season? [Likely]

K. Pricing, Smart Pricing and host-side experiments

86. Design an experiment for a change to a host price-suggestion model. What is randomized, and what is measured? [Likely; 7][12]
    - Hosts may ignore suggestions. How does adoption affect your estimand (intent-to-treat versus compliers)?
    - How do you prevent treatment hosts' price changes from affecting control hosts' bookings?
87. How would you estimate the price elasticity of demand from experiments without running illegal or unfair price discrimination on guests? [Likely]
    - Why do companies like Airbnb generally avoid randomizing the price shown to guests, and what do they do instead?
88. A pricing experiment shows host revenue up 4% in treatment, but marketplace-wide bookings are flat. How do you interpret this? [Likely; 12]
89. How would you test a change in guest fees or the display of total price (for example, showing all-inclusive prices)? Which interference and regulatory concerns apply? [Likely]

L. Fairness, ethics and trust in experimentation

90. How would you check that an experiment does not harm a protected group of guests or hosts, given that you may not observe protected attributes directly? [Likely; 20]
    - How might perceived race or gender be measured under privacy constraints (for example, in Airbnb's Project Lighthouse work), and how would that feed into experiment analysis?
91. Which experiments should not be run at all, or need special review, on a travel/housing marketplace? [Likely]
92. How would you experiment on a trust and safety intervention, such as party prevention or booking restrictions, where the main outcome is a rare, severe harm? [Likely; 17]

M. Past-project deep dive: experimentation probes

93. Tell me about an experiment you ran where the result surprised you. What did you do next? [Likely]
94. Tell me about a time an A/B result was wrong or misleading, for example because of a bug, SRM, interference or novelty. How did you find out? [Likely]
95. Describe a time you pushed back on launching a feature that had a "statistically significant" win. [Likely]
96. Describe a time you changed how your team or org runs experiments, such as a new metric, a new method or new tooling. What was the impact? [Likely]
97. What was the smallest effect you ever detected reliably, and what did you do to get enough sensitivity? [Likely]
98. How did you decide between running an experiment and shipping on offline evidence? Give a time you chose not to A/B test. [Likely]
99. How do you work with data scientists, PMs and engineers when you disagree about how to read an experiment? [Likely]

N. Quick-fire concept checks (likely as follow-ups)

100. What is the difference between statistical significance and practical significance? How do you set the latter? [Likely]
101. Explain Type I and Type II errors, and the cost of each for a ranking launch at Airbnb. [Likely]
102. When would you use a one-sided test? [Likely]
103. What is the stable unit treatment value assumption (SUTVA), and where does it fail at Airbnb? [Likely; 13]
104. Why is the t-test usually fine for non-normal metrics at Airbnb scale, and when does it break? [Likely]
105. What is the difference between relative and absolute lift, and how do you build a confidence interval for relative lift? [Likely]
106. What is a holdout group versus a control group? [Likely]
107. What is the difference between a cluster-randomized design's unit of randomization and its unit of analysis, and how do you compute the standard errors? [Likely]
108. What is Twyman's law, and how does it apply to experiment review? [Likely]

References

None of the sources below could be opened while writing this chapter. The web-search budget was used up and WebFetch could not resolve medium.com or igotanoffer.com. Each entry is listed from prior knowledge. Where the exact article URL was not known with confidence, the entry gives the title and points to the publication's index instead of a guessed link. Confirm every reference before relying on it.

1. "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
2. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD 2019. https://arxiv.org/abs/1810.09591
3. "Scaling Airbnb's Experimentation Platform" (describes ERF, the Experiment Reporting Framework), Airbnb Tech Blog, around 2017. URL not verified. Search via https://medium.com/airbnb-engineering
4. "Designing Experimentation Guardrails", Airbnb Tech Blog, around 2021-2022. URL not verified. Search via https://medium.com/airbnb-engineering
5. Haldar et al., "Improving Deep Learning for Airbnb Search", arXiv / KDD 2020. https://arxiv.org/abs/2002.05515
6. "How Airbnb Measures Future Value to Standardize Tradeoffs", Airbnb Tech Blog, 2022. URL not verified. Search via https://medium.com/airbnb-engineering
7. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219830
8. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Tech Blog (Medium), 2022. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
9. "Airbnb Interview Process & Questions", IGotAnOffer, 2023-2024. https://www.igotanoffer.com/blogs/tech/airbnb-interview-process
10. "Airbnb Machine Learning Engineer Interview Questions", Glassdoor (candidate reports), ongoing. https://www.glassdoor.com/Interview/Airbnb-Machine-Learning-Engineer-Interview-Questions-EI_IE391850.0,6_KO7,32.htm
11. "How Airbnb Achieved Metric Consistency at Scale" (Minerva metrics platform), Airbnb Tech Blog, 2021. URL not verified. Search via https://medium.com/airbnb-engineering
12. Holtz, Lobel, Liskovich and Aral, "Reducing Interference Bias in Online Marketplace Pricing Experiments" (Airbnb collaboration), working paper / arXiv, 2020. URL not verified. Search the title on arXiv or SSRN.
13. Johari, Li, Liskovich and Weintraub, "Experimental Design in Two-Sided Platforms: An Analysis of Bias", Management Science, 2022 (arXiv preprint 2020). URL not verified. Search the title on arXiv.
14. "Artificial Counterfactual Estimation (ACE): Machine Learning-Based Causal Inference at Airbnb", Airbnb Tech Blog, around 2021-2022. URL not verified. Search via https://medium.com/airbnb-engineering
15. Deng, Xu, Kohavi and Walker, "Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data" (CUPED), WSDM 2013. URL not verified. Search the title in the ACM Digital Library.
16. Grbovic and Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219885

17. "Fighting Financial Fraud with Targeted Friction", Airbnb Tech Blog, 2018. URL not verified. Search via https://medium.com/airbnb-engineering
18. Johari, Pekelis and Walsh, "Always Valid Inference: Bringing Sequential Analysis to A/B Testing", arXiv 2015 / Operations Research 2022. URL not verified. Search the title on arXiv.
19. "Selection Bias in Online Experimentation" (winner's curse correction), Airbnb Tech Blog, around 2019. URL not verified. Search via https://medium.com/airbnb-engineering
20. Airbnb "Project Lighthouse" technical paper and posts on measuring discrimination with perceived race under privacy constraints, Airbnb, 2020. URL not verified. Search via https://medium.com/airbnb-engineering or the Airbnb Newsroom.
21. Kohavi, Tang and Xu, "Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing", Cambridge University Press, 2020. General background for SRM, A/A tests, Twyman's law and OEC questions. URL not verified.

05. ML Fundamentals and Modeling Depth: Ranking, GBDT vs. NN, Embeddings, Calibration, Features, Offline/Online Gap

> **Research note (read first).** The shared web-search budget for this run was already used up before this chapter began, so no new searches were possible. Direct fetches failed with DNS errors for every site tried except github.com: arxiv.org, medium.com, teamblind.com, igotanoffer.com, interviewquery.com and trueinterview.io. The only candidate-report material that could be read live is a GitHub-hosted compilation of TrueInterview data: an Airbnb MLE process write-up built from 3 candidate experiences, plus an Airbnb question index [1][2]. Two general format pages were also read [3][4]. The Airbnb papers and blog posts in References [5]-[14] are cited from prior knowledge as the public source of the systems that the [Likely] questions probe. They were **not re-fetched**. Confirm every link before relying on it.
>
> **Tags.** [Reported] means a fetched source attributes the prompt or topic to an Airbnb MLE interview. The citation says which source. [Likely] means an inference from the role, the Senior level and the systems Airbnb has published. A citation next to a [Likely] tag points to the paper behind the topic. It does **not** mean anyone was asked the question.

Format notes

- **There is no standalone "ML fundamentals / ML theory" round in reported Airbnb MLE loops.** The fetched candidate-report summary says no reported round asked for derivations, loss-function proofs or architecture comparisons. "Paper-level theory" and naming a fashionable model without a metric are listed as *not tested* [1].
- **Where fundamentals actually show up:** as depth probes inside the **ML system design** round (one or two rounds, 45-60 min each, run by senior/staff MLEs, described as the core and decisive round of the track). They also show up in the **experience / project deep dive** (45-60 min, a single past project). ML design prompts are sometimes folded into the 45-60 min technical phone screen too [1].
- **Expected structure in those rounds:** framing → label definition → data sources → features → model choice → offline evaluation → serving, latency, freshness, cold start for new listings → experimentation. Model choice took little time in the reported rounds. **Jumping straight to the model is the most-cited failure** [1].
- **Recurring reported follow-up themes:** turning an offline metric into an online one, spatial features such as geohashing, and framing a target as regression vs. bucketized classification. In the lifetime-value prompt the interviewer pushed toward bucketized classification [1].

- **Senior (L5) bar:** the problem scope you owned matters more than model sophistication [1]. Expect "why" questions on every choice: why this label, why GBDT or NN, why this metric, and what you would expect to move online. Questions in this chapter are written as those depth probes. Practise answering each one in 2-4 minutes, tied to an Airbnb surface.
- **Airbnb context that interviewers know well:** search moved from GBDT to deep NNs [5][6]. In-session listing embeddings [7][8], multi-objective journey ranking [10], learning to rank diversely [9], a customized pricing regression loss [11] and interleaving for ranking experiments [12] are all public. Expect probes that check whether you can reason about the same trade-offs.

Questions

A. Problem framing, labels and targets

1. You are asked to estimate the lifetime value of a listing (or property). Would you frame it as regression or as bucketized classification, and why? [Reported] [1][2]
    - The interviewer pushes you toward bucketized classification. What do you gain and lose?
    - How do you choose bucket boundaries, and how do you turn bucket probabilities back into an expected value?
    - How do you handle the heavy right tail and listings that churn off the platform (censoring)?
2. How would you identify "high-quality listings"? What is your label, given that no ground truth exists? [Reported] [1]
    - Which proxies (reviews, rebook rate, cancellations, CS tickets) would you combine, and how do you avoid circularity with the ranking model's own exposure?
3. Choose a listing's cover photo to lift bookings. Is this a supervised problem, a ranking problem or a bandit problem? [Reported] [1]
    - What label would you train on, given that the cover photo you showed also caused the clicks?
    - How do you handle listings with only a few photos or a few impressions?
4. Detect relisting (a host or listing removed for policy reasons reappears under a new account). What features and model family would you use? [Reported] [1][2]
    - Is this classification, similarity search or entity resolution? Defend the framing.
    - How do you choose an operating threshold when false positives hurt legitimate hosts?
5. For search ranking, what is the training label: click, long click, wishlist, booking request, accepted booking or completed stay without cancellation? [Likely] [5][10]
    - How do you weight these in a single objective vs. a multi-task model?
    - How do you deal with label delay (booking days after search)?
6. Rank home search results when the user gives no text query (only location and dates, or none at all). What does "relevance" mean here, and what is the label? [Reported] [2]
7. How would you define a negative example in Airbnb search: an impressed and not clicked listing, a clicked and not booked listing, or a random listing? [Likely] [5][7]

    - What bias does each choice introduce?
8. A booking requires host acceptance. How does that change your label definition and your model? [Likely] [5]
9. When would you predict a calibrated probability rather than a ranking score, and which downstream consumers in Airbnb need which? [Likely]
10. You have far fewer bookings than clicks. How does label sparsity drive your choice of objective, model capacity and evaluation? [Likely] [5]

B. Learning to rank

11. Compare pointwise, pairwise and listwise learning-to-rank objectives. Which would you start with for Airbnb search, and why? [Likely] [5]
    - Airbnb's search team reported success with a pairwise formulation. Why might pairwise suit a booking-or-not label inside a search?
12. How do you build training pairs or lists from search logs (booked listing vs. the other listings impressed in the same search)? [Likely] [5]
    - How many negatives per positive, and does sampling them change the metric you are optimizing?
13. Explain how LambdaRank / LambdaMART connects a pairwise loss to NDCG. [Likely]
    - Why can't you optimize NDCG directly with gradient descent?
14. What offline metric would you use for search ranking (NDCG on the booked listing, MRR, recall@k, AUC)? Why? [Likely] [5]
    - Why might AUC improve while NDCG is flat?
15. How do you handle position bias in logged search data? [Likely] [6]
    - Compare including position as a feature (with dropout or set to a constant at inference), inverse propensity weighting and randomization-based estimates.
    - How would you estimate position propensities at Airbnb without hurting users?
16. Your ranker learns that cheap listings get booked. Why does pointwise use of price fail, and how would you encode price relative to the user's query and market? [Likely] [6]
17. How do you rank diversely (avoid ten near-identical listings) without destroying relevance? [Likely] [9]
    - Is diversity a re-ranking step, a loss term or a list-aware model? Trade-offs?
    - How would you measure whether diversity helped bookings?
18. How would you design a two-stage system (retrieval + ranking) for Airbnb search, and what does each stage optimize? [Likely] [13]
    - How do you avoid the ranker never seeing listings that retrieval filters out (selection bias)?
19. Airbnb search is a multi-step journey (search → click → listing page → request → booking). How would you model it with multi-task learning? [Likely] [10]
    - How do you combine task heads into a single ranking score?
    - How do you handle tasks with very different base rates and loss scales?
20. How would you incorporate host-side objectives (acceptance likelihood, host preferences, fairness of exposure) into the ranking score? [Likely] [5]
21. Personalized search ranking: which user signals would you use, and how do you avoid overfitting to a user's last click? [Reported] [2]
22. How would you rank Airbnb Experiences differently from Stays, given much less data per item? [Likely] [8]

C. GBDT vs. neural networks

23. When would you choose GBDT over a deep neural network for a tabular ranking problem, and vice versa? [Likely] [5]
    - Airbnb moved search from GBDT to a NN and initially saw little gain. Why does that happen, and what made the NN eventually win?
24. A NN on the same features as your GBDT performs worse. What do you check first? [Likely] [5]
    - Feature normalization, distribution shape, learning rate, capacity, or label noise: how do you tell which?
25. GBDTs are invariant to monotone feature transforms; NNs are not. What normalization would you apply to price, review count and distance before a NN? [Likely] [5]
    - Why might a log transform or a Gaussian rank transform matter for the NN but not for the trees?
26. How would you use a GBDT and a NN together (GBDT leaves as NN features, stacking, or distillation)? [Likely]
27. Why might adding listing-ID embeddings to the NN overfit at Airbnb, while the same idea works for e-commerce item IDs? [Likely] [5]
    - What about the booking-per-listing distribution makes it hard?
28. Deep models are harder to interpret. How do you explain to a PM or a host why a listing ranked lower? [Likely] [5]
    - Compare SHAP, partial dependence plots and ablation-based feature importance. Which are trustworthy for a NN?
29. Your NN is more accurate offline but has 3x the inference latency. How do you decide, and what can you do to close the gap? [Likely]
30. How do you set the main GBDT hyperparameters (depth, learning rate, number of trees, min child weight), and which ones matter most for overfitting? [Likely]
31. What kinds of feature interactions does a GBDT capture well, and which ones does it capture badly? [Likely]
32. How would you handle a monotonicity requirement (for example, ranking must not reward higher price, all else equal)? [Likely] [6]

D. Embeddings and representation learning

33. How would you learn listing embeddings from user click sessions? [Likely] [7]
    - Why treat a session as a "sentence" and use skip-gram with negative sampling?
    - What is the effect of using the booked listing as global context?
    - Why sample negatives from the same market, and what goes wrong if you sample uniformly?
34. How do you produce embeddings for a brand-new listing with no sessions (cold start)? [Likely] [7]
    - Averaging embeddings of nearby, similar listings: what are the failure modes?
35. Bookings are too sparse to learn per-user embeddings. How would you learn user-type and listing-type embeddings in the same space? [Likely] [7]
36. How do you evaluate listing embeddings offline before they reach production? [Likely] [7]

    - What intrinsic checks (geography, price, architecture style) and which extrinsic checks would you run?
37. Use embeddings for real-time personalization: how do you turn a user's in-session clicks and skips into ranking features? [Likely] [7]
    - How do you weight recent clicks vs. older ones, and how do you use negative signals (skipped listings)?
38. Design user embeddings for semantic search (free-text queries such as "cabin with a hot tub near a lake"). [Reported] [2]
    - How would you align text queries and listings in the same embedding space?
    - Which training pairs and hard negatives would you use?
39. How would you build embedding-based retrieval for Airbnb search, and how do you combine it with hard filters (dates, price, map bounds, guests)? [Likely] [13]
    - Pre-filter vs. post-filter with ANN: what are the recall and latency trade-offs?
40. Two-tower vs. cross-encoder: where in the stack would you use each, and why? [Likely] [13]
41. How do you keep embeddings fresh when listings change (price, photos, availability) daily? [Likely]
    - What breaks downstream when you retrain the embedding model and the space rotates?
42. How do you handle spatial information: geohashing, learned location embeddings or raw latitude/longitude? [Reported] [1]
    - Why can raw lat/long be a poor NN input, and how do geohash granularity choices affect generalization?
43. How would you use image embeddings from listing photos (amenity detection, room type, photo quality) as ranking features? [Likely] [14]
44. When does a pre-trained LLM / foundation-model embedding beat a domain-trained embedding for listings, and when does it not? [Likely]

E. Calibration and probability quality

45. What is calibration, and why does it matter if you only use the score to rank? [Likely]
    - Name two Airbnb consumers where a calibrated probability is required (for example combining multi-task heads, expected-value pricing, fraud thresholds).
46. How do you measure calibration (reliability diagram, ECE, log loss vs. Brier score)? [Likely]
47. Compare Platt scaling, isotonic regression and temperature scaling. When would each fail? [Likely]
48. You down-sample negatives 10:1 for training. How do you correct the predicted probabilities? [Likely] [5]
49. Your model is well calibrated overall but miscalibrated for new listings and small markets. How do you detect and fix that? [Likely]
50. You combine pClick x pBook|click x pAccept into one ranking score. What happens if one head is miscalibrated? [Likely] [10]
51. Calibration drifts after a ranking change, because the model now trains on its own exposure. How do you monitor and recalibrate? [Likely]
52. For a fraud or trust model, how do you choose the decision threshold, and how do you communicate the precision-recall trade-off? [Likely]

F. Feature engineering

53. Name the 10 most important feature families for Airbnb search ranking, and justify each. [Likely] [5]
    - Which ones are query-dependent (dates, guest count, location) and which are listing-only?
54. How would you build a feature for "price relative to the market for these dates and this listing type"? [Likely] [6][11]
55. How do you engineer location features: distance to the searched point, neighbourhood popularity, geohash-level booking rates? [Reported] [1]
56. How do you create features from reviews (count, average, recency, text) without leaking future information? [Likely]
57. How do you avoid target leakage when computing historical listing CTR or booking rate as a feature? [Likely]
    - Point-in-time correctness: how do you guarantee it in your training data pipeline?
58. How do you handle new listings whose historical features are missing? Imputation, a missingness indicator or a separate model? [Likely] [6]
    - Airbnb has written about new listings being disadvantaged in search. How would you estimate their engagement?
59. How do you handle high-cardinality categorical features (city, host ID, amenity sets) in a GBDT vs. a NN? [Likely]
60. Which features would you compute in real time (session clicks, current search filters), and which in batch? What does each choice cost? [Likely] [7]
61. How do you detect that a feature is computed differently at serving time than in training (training-serving skew)? [Likely]
62. How do you decide whether a new feature is worth its engineering and latency cost? [Likely]
    - What ablation and online test would you run?
63. Seasonality and lead time strongly affect demand. How would you encode check-in date, lead time and length of stay? [Likely] [11]
64. How would you build features capturing host quality and host responsiveness? [Likely] [5]
65. How do you handle features that the ranking model itself influences (for example, a listing's CTR depends on how highly it was ranked)? [Likely] [6]

G. Offline evaluation and the offline/online gap

66. How do you translate an offline metric (NDCG, AUC) into an expected online metric (bookings, revenue)? [Reported] [1]
    - Have you ever validated the correlation between your offline metric and online lift across past launches?
67. Offline NDCG improved by 2%, but the online A/B test showed no booking lift. List the hypotheses you would investigate, in order. [Likely] [5]
    - Training-serving skew, logging bias, metric mismatch, a novelty effect, the experiment being underpowered, or interference between listings: how do you rule each out?

68. Why can a model trained on logs from the old ranker look good offline but fail online (feedback loops, counterfactual evaluation)? [Likely] [6]
    - How would off-policy evaluation (IPS, doubly robust) help, and what assumptions does it need?
69. How do you build an offline evaluation set that represents the online traffic mix (markets, devices, new vs. returning users)? [Likely]
70. What is a "replay" or "re-rank the logged results" evaluation, and what are its blind spots? [Likely]
71. When would you use interleaving instead of an A/B test to compare two rankers? [Likely] [12]
    - What does interleaving measure well, and what can it not tell you (for example, revenue or long-term effects)?
    - How do you validate that interleaving outcomes agree with A/B outcomes?
72. Your model wins in aggregate but loses in some markets or segments. How do you decide whether to launch? [Likely]
73. How do you guard against overfitting to the offline evaluation set after many iterations? [Likely]
74. How do you estimate the online impact of a model before running an A/B test, to prioritize which experiment to run? [Likely]
75. How would you set up a long-term holdout for ranking changes, and what would you learn from it that an A/B test cannot show? [Likely]
76. What metric would you use when the goal is "a great stay", not just a booking (cancellations, reviews, rebookings)? [Likely] [10]

H. Training, generalization and debugging

77. Walk through how you would detect and fix overfitting in a deep ranking model trained on one year of search logs. [Likely] [5]
78. How do you split data for time-dependent problems (random vs. temporal vs. by user/listing), and why does it matter at Airbnb? [Likely]
79. Your training loss decreases but validation NDCG oscillates. What could cause it? [Likely]
80. How do you handle class imbalance (bookings much rarer than impressions): re-weighting, sampling or focal loss? [Likely]
    - Which of these break calibration?
81. How often would you retrain the ranker, and how do you decide (fixed schedule vs. drift-triggered)? [Likely]
82. Model quality dropped after a holiday season. How do you diagnose concept drift vs. data pipeline bugs? [Likely]
83. How do you make training reproducible enough that a 0.5% offline gain is real and not noise? [Likely]
    - How many seeds or bootstrap resamples would you run before trusting a gain?
84. How do you choose the loss for a pricing or demand model where under- and over-prediction have asymmetric costs? [Likely] [11]
85. How would you use dropout, early stopping and weight decay together, and how do you choose among them? [Likely]

I. Regression, forecasting and pricing-flavoured modeling

86. Airbnb proposed a customized regression loss for price suggestions, because true "optimal price" labels do not exist. How would you frame learning a price when you only observe booked or not booked at a set price? [Likely] [11]
87. How would you estimate demand elasticity when price is confounded with listing quality and seasonality? [Likely] [11]
88. How would you evaluate a price-suggestion model offline when hosts can ignore the suggestion? [Likely] [11]
89. How would you forecast occupancy or demand for a listing 90 days out, and which uncertainty estimates would you provide? [Likely]
90. Quantile regression vs. mean regression: when would you use each for host-facing estimates (earnings, nightly price)? [Likely]

J. Classification, trust and quality models

91. Design the label and model for detecting fake or fraudulent listings, given that positives are rare and arrive with delay. [Likely]
    - How do you handle labels that only exist for cases that an earlier model or rule flagged (selection bias)?
92. How do you evaluate a classifier when precision matters far more than recall (for example, removing a host's listing)? [Likely]
93. How would you use text (listing description, messages) in a classifier: TF-IDF + GBDT, fine-tuned transformer, or LLM embeddings? Trade-offs? [Likely]
94. How do you build an active-learning or human-in-the-loop loop to label listing categories or quality? [Likely] [14]
95. How would you check a ranking or trust model for unfair outcomes across host or guest groups? [Likely]

K. Experimentation-adjacent modeling (role focus)

96. Your new ranker is evaluated by user-randomized A/B. Why can supply competition between listings bias the estimate, and how does that affect how you interpret a model's lift? [Likely] [12]
97. How would you use a model-based surrogate metric (predicted booking value) to read experiments faster? What validation does the surrogate need? [Likely]
98. How would you use pre-experiment covariates (CUPED or regression adjustment) to reduce variance, and which model features make good covariates? [Likely]
99. Heterogeneous treatment effects: how would you model which users or markets benefit from a ranking change (uplift models, causal forests)? [Likely]
100. How do you decide between a contextual bandit and a supervised model plus A/B test for something like cover-photo selection? [Likely] [1]
101. If you can only run one experiment this quarter, how do you choose among three model improvements using offline evidence? [Likely]

L. Senior-level depth probes (from the deep dive and design rounds)

102. In your most impactful model, which single modeling decision mattered most, and how do you know? [Likely] [1]
103. Tell me about a time a simpler model (heuristic, logistic regression, GBDT) beat a more complex one in production. Why did it win? [Likely] [1]

104. What did you choose *not* to model, and why? [Likely]
105. How did you choose your offline metric, and did it ever mislead you? [Likely] [1]
106. If I gave you twice the data vs. twice the features vs. twice the model capacity, which would you take for Airbnb search, and why? [Likely]
107. How do you explain a ranking model's trade-offs to a PM who only cares about bookings? [Likely]

References

[1]-[4] were fetched live for this chapter. [5]-[14] are cited from prior knowledge and were **not** fetched, because DNS resolution failed for arxiv.org, medium.com and other non-GitHub hosts. Verify those links before relying on them.

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview data, built from 3 candidate-reported experiences, CC BY 4.0), GitHub: kevin-2023-code/Tech-Interview-Questions, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb" company question index (lists reported prompts incl. "Listing Lifetime Value — Estimation", "Design User Embedding Semantic Search", "Rank Home Search Results Without a Text Query", "Design A Personalized Search Ranking System", "Design Airbnb Relisting Detection"), GitHub: kevin-2023-code/Tech-Interview-Questions, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
3. "Interview formats of top companies" (Airbnb section), Tech Interview Handbook, GitHub: yangshun/tech-interview-handbook, ongoing. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
4. "Airbnb" company prep page (confirms no ML-specific section; general SWE loop), GitHub: jugaldb/faang-interview-prep, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
5. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD 2019. https://arxiv.org/abs/1810.09591
6. Haldar et al., "Improving Deep Learning for Airbnb Search", arXiv / KDD 2020. https://arxiv.org/abs/2002.05515
7. Grbovic & Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
8. Grbovic, "Machine Learning-Powered Search Ranking of Airbnb Experiences", Airbnb Tech Blog (Medium), 2019. https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789
9. Abdool, Haldar et al., "Learning To Rank Diversely At Airbnb", arXiv, 2022. https://arxiv.org/abs/2210.07774
10. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning", arXiv / KDD 2023. https://arxiv.org/abs/2305.18431
11. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219830
12. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Tech Blog (Medium), 2022. https://medium.com/airbnb-engineer

ing/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
13. "Embedding-Based Retrieval for Airbnb Search", Airbnb Tech Blog, 2025. The article URL was not verified. Search for the title via https://medium.com/airbnb-engineering
14. Airbnb Tech Blog posts on listing-photo amenity detection / room classification (2018-2019) and on building Airbnb Categories with ML and human-in-the-loop (2022-2023). The article URLs were not verified. Search for the titles via https://medium.com/airbnb-engineering

06. General Backend System Design: Booking, Availability, Search, Messaging, Payments and Feature Store

> **Research note.** This session's web-search budget ran out, and most interview sites (Glassdoor, Blind, LeetCode, Reddit, Medium, IGotAnOffer, TrueInterview, the Airbnb Tech Blog) could not be reached. The chapter therefore rests on the sources that did load: aggregated candidate reports for 2025 to 2026 hosted on GitHub [1][2][3][4], a community prep guide [5], a crowd-sourced question list [6], the Tech Interview Handbook [7] and Airbnb's open-source Chronon repository [8]. A question is tagged [Reported] only when one of these sources reports it for Airbnb. Every other question is [Likely]: an inference from Airbnb's products and the senior bar described in those sources. Before the onsite, re-run searches on Glassdoor, Blind and LeetCode Discuss for "Airbnb system design 2026".

Format notes

- **Rounds and length.** Candidates report one or two system design rounds of 45 to 60 minutes each. A second design round is sometimes added late in the process when the signal is borderline, and one candidate said that extra round decided the outcome [1][2]. The Tech Interview Handbook lists one "System Design/architecture" round in the general engineering onsite [7]. For the 1-to-3-year level (G8), one report describes a 60-minute design screen plus a 60-minute onsite design round on Miro or Excalidraw [5].
- **Interviewer.** A senior or staff engineer runs the round. MLE loops usually have a separate ML system design round (see chapter 03), but generic backend prompts also appear in engineering loops at senior level [1][2][3].
- **Senior bar.** The aggregated reports say design depth "reads staff-level even for senior candidates." Candidates are expected to "drive requirements yourself, fix a minimal scope in the first few minutes, then go deep where the interviewer leans in" [1].
- **Common failure.** Trying to cover every feature. One candidate lost an in-memory SQL engine round by attempting the whole feature set without first agreeing what "done" meant. The reports say design rounds "reward scoping and negotiated trade-offs" [1].
- **Recurring themes.** Double-booking prevention, availability search, and hot partitions caused by a big event [1][5]. Others are notification fan-out, chat, job scheduling, key-value stores pushed onto CAP trade-offs, and streaming aggregation with Flink and Kafka [1][3][4][6].

- **What this means for an MLE.** Expect a classic backend prompt framed around Airbnb's own products (booking, calendar, search, messaging). Expect the interviewer to probe whatever your background makes natural: feature stores, logging pipelines and experiment assignment.

Questions

A. Booking and reservation system

1. Design Airbnb's booking system: guests search for available listings and book them, and the system must never double-book a listing. [Reported] [1][3][5]
    - What are the functional requirements you would commit to in the first five minutes, and what would you explicitly leave out?
    - Is a booking request a strongly consistent write? Which parts of the flow can be eventually consistent?
    - How do you guarantee two guests cannot book overlapping dates on the same listing at the same moment?
    - Would you use pessimistic row locks, optimistic concurrency with version numbers, or a uniqueness constraint on (listing, night)? Why?
2. Design a booking platform that prevents double bookings and also supports search. Follow-up: a big event (for example a festival or a World Cup final) creates a hot partition. How do you handle it? [Reported] [5]
    - Which keys get hot: listing IDs, a city's search shard or the availability index?
    - How do you shed or queue load without letting someone book a listing that is already taken?
    - Would you pre-shard or add caches for a known upcoming event? How would you detect an unplanned spike?
3. Design a hotel booking system (rooms with inventory counts rather than unique homes). [Reported] [3]
    - How does inventory-count booking differ from Airbnb's one-unit-per-listing model?
    - How do you handle overbooking policies, if any are allowed?
4. Walk through the state machine of a reservation: request, pending host approval, accepted, paid, checked in, completed, cancelled or altered. [Likely]
    - Where do you persist state transitions, and how do you make them auditable?
    - How do you expire a booking request the host never answers within 24 hours?
5. How would you implement a temporary hold on dates while the guest is on the checkout page? [Likely]
    - How long should the hold last, and what releases it?
    - What happens if the payment succeeds after the hold has expired?
6. Design Instant Book versus Request to Book. How do the two flows differ in consistency and latency requirements? [Likely]
7. How do you handle reservation alterations, such as changing dates or guest count on an existing booking? [Likely]
    - How do you avoid releasing the old dates before the new dates are secured?
8. Design the cancellation flow, including refunds that depend on the cancellation policy and the time remaining before check-in. [Likely]

9. Design a waitlist system, for example for a sold-out experience or a high-demand listing. [Reported] [3]
    - How do you notify the next person fairly and with a deadline to claim?
    - How do you avoid a thundering herd when a slot opens?
10. How would you shard the reservations database as Airbnb grows across regions? [Likely]
    - Shard by listing, by guest or by host? Which queries become cross-shard?
11. How do you make the "Reserve" API idempotent when the mobile client retries on a flaky network? [Likely] [9]
12. Booking needs data from several services: listing, pricing, availability, payments and trust. How do you coordinate the transaction across them? [Likely] [9]
    - Would you use a saga, two-phase commit or an orchestrator? What are the compensating actions?

B. Availability calendar

13. Design the availability calendar service for millions of listings, in which hosts block dates and set minimum and maximum nights and per-night prices. [Likely] [1][3]
    - What is the data model: one row per listing-night, interval ranges or bitmaps?
    - What are the read-to-write ratios for search, listing-page views and host edits?
14. Design Google Calendar (events, recurrence, invites, free/busy). [Reported] [3]
    - How do you store and expand recurring events efficiently?
    - How would you answer "is everyone free from 2 to 3pm" across many calendars?
15. Hosts sync calendars with other platforms through iCal. How do you import and export them without causing double bookings? [Likely]
    - What polling frequency would you use, and how do you handle conflicts between an external block and a pending Airbnb request?
16. How do you answer "which listings in Paris are available from June 3 to June 7 for 4 guests" at search time? [Likely] [1]
    - Do you pre-compute availability into the search index, or filter after retrieval?
    - How stale can the search index's availability be? What happens if a user clicks a listing that has just become unavailable?
17. Design the split-stay feature, which offers two listings that together cover a guest's dates when no single listing does. [Likely] [1]
    - How do you search pairs of listings without an O(n squared) blow-up?
    - What changes if you allow three-way splits?
18. How do you handle time zones and check-in/check-out boundaries for listings around the world? [Likely]
19. A host edits the calendar while a guest is booking the same dates. Which write wins, and how do you guarantee correctness? [Likely]
20. How would you propagate availability changes to search, pricing and ML ranking features within seconds? [Likely]
    - Would you use change data capture (CDC) to a log, dual writes or an outbox pattern?

C. Search and discovery backend

21. Design the Airbnb home page: search, availability and ranking. [Reported] [3]

    - What are the latency budgets for retrieval, filtering and ranking?
    - How does the page behave when the ranking service is slow or down?
22. Design ranking for home search results when the user has typed no text query (only location, dates and guests). [Reported] [3]
23. Design a query system over user activity that filters by time window and geography. [Reported] [2][3]
    - What index structures would you use for the geo part (geohash, S2 cells, quadtrees) and for the time part?
    - How do you support queries such as "views in this bounding box in the last 7 days" at scale?
24. Design a map-based search: as the user pans and zooms, return the listings within the viewport. [Likely]
    - How do you avoid hammering the backend on every pan event?
    - How do you cluster pins when thousands of listings fall in the viewport?
25. How would you build the search index (inverted index plus geo index) and keep it fresh as listings, prices and availability change? [Likely]
    - Full reindex versus incremental updates: what triggers each?
26. Design autocomplete for destinations and points of interest. [Likely]
27. Design semantic search over user and listing embeddings. [Reported] [3]
    - How do you choose and operate an approximate nearest-neighbor index (HNSW, IVF-PQ) next to structured filters?
    - How do you re-embed and re-index without downtime?
28. Design the host listings page, which aggregates data from many services for hosts with hundreds of listings, and optimize it. [Reported] [3]
    - What would you pre-compute, cache or paginate?
    - How do you avoid N+1 calls across service boundaries?
29. Design pagination for search results that stays stable when ranking or availability changes between page loads. [Likely]
30. How would you cache search results, given that the same location and dates are queried by many users but prices and availability change? [Likely]
31. Design the wishlist service, including shared wishlists among group members. [Likely]

D. Messaging and notifications

32. Design a group chat system (WhatsApp-like). [Reported] [5]
    - How do you guarantee per-conversation ordering and exactly-once display?
    - How do you deliver to offline users and sync across multiple devices?
33. Design group chat with a feature that returns a user's top-N most active chats. [Reported] [1]
    - How would you define and maintain "active" in real time?
34. Design a Slack-like chat system. [Reported] [3][4]
35. Design guest-host messaging on Airbnb: messages tied to a listing or reservation, with read receipts and translation. [Likely]
    - How do you store the conversation so it is linked to the reservation and visible to support agents?
    - Where would you hook in fraud or off-platform-payment detection without adding latency to sending?

36. Design a unified notification platform that handles both event-triggered and batch sends. [Reported] [1][3][4]
    - How do you prioritize transactional notifications (booking confirmed) over marketing?
    - How do you de-duplicate, rate-limit per user and respect quiet hours across time zones?
    - What happens when a downstream provider (APNs, FCM, an SMS gateway) is slow or failing?
37. How would you design per-user notification preferences and channel fallbacks (push, then email, then SMS)? [Likely]
38. Design a customer-support ticketing system that routes tickets to the right agent queue. [Reported] [3]
    - How do you combine rules-based routing with an ML classifier, and how do you fall back when the model is down?
39. Design a system that generates customer-support instructions from company policies, for example with an LLM. [Reported] [3]
    - How do you keep generated instructions in sync when a policy changes?
    - How do you evaluate correctness before showing them to agents?
40. Design a system that sends reminder messages (check-in instructions, review requests) at specific times relative to each reservation. [Likely]

E. Payments, payouts and financial correctness

41. How would you design the payment flow so a guest is never charged twice, even with retries and partial failures? [Likely] [9]
    - What is an idempotency key? Who generates it, and how long do you keep it?
    - How do you separate the local DB transaction from the remote payment-provider call?
42. Design host payouts: scheduling, splitting among co-hosts, multiple currencies and payout methods. [Likely]
43. Design refunds across multiple payment methods and dates, honoring a priority order. [Likely] [10]
44. Design a ledger (double-entry) for Airbnb's payments platform. [Likely]
    - How do you reconcile your ledger against the payment processor's settlement files?
45. Design a banking-like service with deposit, withdraw, transfer and balance. Follow-up: how would you scale it into a distributed service? [Likely] [1]
46. Design a currency-conversion and price-display service that shows consistent prices from search through checkout. [Likely]
    - How do you lock an exchange rate for a quote?
47. Design a retryer or retry library for calls to unreliable downstream services. [Likely] [1]
    - How do you decide which errors are retryable?
    - How do you combine exponential backoff, jitter, deadlines and circuit breakers?
48. How would you design fraud checks into the payment path without hurting checkout latency? [Likely]

F. Data, logging and analytics infrastructure

49. Design a highly interactive data analysis system. [Reported] [6]
    - What latency problem appears if you start with batch Spark processing?
    - How would you use Flink with Kafka for streaming aggregation?
    - How does recovery work when a Flink job fails (checkpoints, offsets, exactly-once)?
50. Design an ad-click (or listing-click) event aggregation system. [Reported] [3]
    - How do you handle late and duplicate events?
    - How do you reconcile real-time counts with batch-corrected counts?
51. Model cumulative booking metrics, including the first and last booking times per user or listing. [Reported] [3]
    - Would you compute this incrementally or recompute it? How do you backfill when the logic changes?
52. Design the logging pipeline that captures search impressions, clicks and bookings for ranking training data. [Likely]
    - How do you join impressions to bookings that happen days later?
    - How do you guarantee the logged features match what the model saw at serving time?
53. Design a metrics and dashboard system for marketplace health (bookings, nights, supply) with daily and real-time views. [Likely]
54. How would you design data-quality checks and alerting for core tables that feed models and experiments? [Likely]

G. Feature store and ML platform infrastructure

55. Design a feature store that serves offline training and low-latency online ranking from one feature definition. [Likely] [8]
    - How do you guarantee point-in-time correctness in training backfills?
    - How do you measure and alert on online/offline consistency?
56. How would you support windowed aggregations (for example a listing's bookings in the last 7 days) in both streaming and batch? [Likely] [8]
    - How do you handle very skewed keys, such as a popular listing or a bot user, in the aggregation?
57. Design the online serving layer of a feature store for a ranking request that fetches features for 1,000 candidate listings in under 50 ms. [Likely] [8]
    - What key-value store would you choose? How do you batch, cache and tolerate tail latency?
58. How do you version features, and what happens to running models when a feature definition changes? [Likely]
59. Design a model-serving platform that supports canary releases, shadow traffic and rollback. [Likely]
60. Design an embedding-serving service that keeps listing embeddings fresh as listings and behavior change. [Likely]
61. Design a job scheduler for about ten million users. [Reported] [1][3][4]
    - How do you guarantee each job runs at least once, and when do you need exactly-once?
    - How do you scale the dispatcher and avoid one hot time bucket (for example midnight UTC)?

62. Design a distributed scheduler for ML training and backfill jobs with dependencies (a DAG). [Likely]

H. Experimentation platform (backend view)

63. Design an A/B testing platform. [Reported] [3]
    - How do you assign users to variants deterministically and consistently across web, iOS and Android?
    - How do you handle logged-out users who later log in?
    - How do you log exposures so analysis is not diluted?
64. Design the assignment service's API and latency budget, given that every page load may evaluate dozens of experiments. [Likely]
    - Do you evaluate on the server or client SDK? How do you push config changes?
65. How would you support mutually exclusive experiment layers and holdouts? [Likely]
66. Design the metrics pipeline that computes experiment scorecards daily for hundreds of experiments and thousands of metrics. [Likely]
67. How would you design a kill switch or automated guardrail that stops an experiment when a key metric degrades? [Likely]
68. Design a network-aware experiment for a "Wi-Fi badge" on listings where treatment effects can spill over between listings. [Reported] [3]
    - How would the platform support cluster or market-level randomization?

I. Generic distributed-systems building blocks (asked at Airbnb)

69. Design a distributed key-value store. Follow-up: walk through CAP trade-offs. [Reported] [1]
    - How do you replicate, and what consistency level do reads get?
    - How do you handle node failure and rebalancing?
70. Design an in-memory SQL engine. Agree on scope first. [Reported] [1]
    - Which subset of SQL (select, where, join, group by) do you commit to, and why?
71. Design an online chess game. [Reported] [3][4]
    - How do you sync game state in real time and handle disconnects and clocks?
72. Design a rate limiter for Airbnb's public APIs and internal services. [Likely]
73. Design a URL shortener for share links to listings and wishlists. [Likely]
74. Design a distributed cache in front of the listing service, and explain invalidation when a host edits a listing. [Likely]
75. Design account merge and user de-duplication when people create multiple accounts. [Likely] [1]
    - How do you merge reservations, reviews and payment methods safely?

J. Other Airbnb product surfaces

76. Design the reviews system: double-blind reviews revealed only when both parties submit or the window closes. [Likely]
    - How do you implement the reveal-at-deadline logic reliably?
77. Design the listing creation and photo upload pipeline, including image processing and moderation. [Likely]
78. Design Experiences booking with time slots, group capacity and a waitlist. [Likely] [3]

79. Design a multi-traveler itinerary service that combines stays and experiences, for example filling a layover with experiences. [Likely] [10]
80. Design a relisting detection system that catches a banned host relisting the same property under a new account. [Reported] [2][3]
    - How do you use geohashing and photo or text similarity at scale?
81. Design a listing lifetime-value estimation service and its batch and online parts. [Reported] [2][3]
82. Design the trust and safety screening pipeline that scores every booking before confirmation. [Likely]
    - How do you keep it off the critical path when possible, and fail open or closed when it is down?
83. Design the host dashboard's earnings and occupancy insights. [Likely]
84. Design Airbnb's internationalization service for translated strings and listing translations. [Likely]

K. Cross-cutting senior probes (apply to any prompt)

85. What are your back-of-the-envelope numbers for this design (QPS, storage, peak-to-average ratio)? [Likely] [1]
86. You have 45 minutes. Which two components would you deep-dive, and why those? [Likely] [1]
87. Where is the single point of failure in what you just drew? [Likely]
88. How do you deploy a schema change to the reservations table with zero downtime? [Likely]
89. How would you run this system across multiple regions? What happens during a region failover in the middle of a booking? [Likely]
90. What metrics, alerts and SLOs would you define for this service? [Likely]
91. Which part of this design would you A/B test, and what could go wrong with the test because of the architecture (caching, sticky assignment, logging gaps)? [Likely]
92. How would you migrate from the existing monolith to this service-oriented design incrementally? [Likely]
93. If traffic grew 10x tomorrow, what breaks first? [Likely]
94. What would you simplify if you had to ship a first version in two weeks? [Likely] [1]
95. How do you secure personal data (addresses, IDs, payment details) in this design, and who can access it? [Likely]

References

Sources 1 to 8 were fetched in this session. Sources 9 and 10 were recalled or listed by another source and were not fetched.

1. "Airbnb Software Engineer Interview Process" (aggregated from 41 candidate reports, 2025 Q4 to 2026 Q2), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
2. "Airbnb Machine Learning Engineer Interview Process", kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Intervie

w-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
3. "Airbnb: every question" (system design titles with report dates, 2025 to 2026), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
4. "Airbnb" company overview (rounds and prompts by round type), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/README.md
5. "Airbnb" company prep page (Jan 2026 senior SWE booking/hot-partition report, G8 loop description), jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
6. "Airbnb interview questions" (Problem 10: data analysis system with high interactivity), warrenzhu25/system-design, GitHub. https://github.com/warrenzhu25/system-design/blob/master/airbnb_interview_questions.md
7. "Interview formats of top companies", Tech Interview Handbook (yangshun), GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
8. "Chronon: a data platform for AI/ML" (Airbnb open-source feature platform), airbnb/chronon, GitHub. https://github.com/airbnb/chronon
9. "Avoiding Double Payments in a Distributed Payments System", The Airbnb Tech Blog (Medium), 2019. Recalled from memory; this session could not reach medium.com, so the URL was not checked. https://medium.com/airbnb-engineering/avoiding-double-payments-in-a-distributed-payments-system-2981f6b070bb
10. Airbnb 1point3acres-sourced coding prompts, including "Refund Allocation by Payment Method and Date Priority" and "Fill Layover Hours by Booking Experiences" (folder listing only), FlowSight/coding, GitHub. https://github.com/FlowSight/coding/tree/main/company%20wise/airbnb/1p3a

07. Past Project / Technical Experience Deep Dive: Reported Prompts and Probing Follow-ups for Senior MLEs

> Research note (read first). The shared web-search budget was already used up when this chapter was written, so no new search queries could be run. The egress proxy also could not resolve Glassdoor, Blind, LeetCode, IGotAnOffer, InterviewQuery, Hello Interview, PracHub, TrueInterview and Medium. GitHub could be reached, so this chapter is built mainly from pages fetched live from GitHub: a mirror of about 100 Airbnb interview posts from 1point3acres (the FlowSight mirror, in Chinese and English) and the TrueInterview Airbnb guides (CC BY 4.0), which are based on 3 MLE reports and 41 SWE reports from 2025 Q4 to 2026 Q2. References [1]-[29] were fetched and read. [30]-[33] are cited from prior knowledge and were not re-fetched. Tags: **[Reported]** means a cited source says a candidate was asked this, or something very close to it, at Airbnb. **[Likely]** means an inference from the role, the senior bar and Airbnb's products. It does not mean anyone reported it. Most of the deep-dive reports come from SWE and infrastructure loops, but the MLE guide says the MLE track runs the same round format [1].

Format notes

Aspect	What sources report
Name on the schedule	"Experience deep dive", "Technical Experience Interview", "Project Deep Dive", "Resume deep dive", "Project presentation", or for ML roles "ML Experience" [1][2][6][7][9][16][17]
Length	45-60 min. One candidate's round was booked for 1 hour and ran about 90 minutes [1][8][9][14]
Interviewer	Usually a senior IC or an engineering manager, and sometimes a senior manager from a neighbouring team (staff loops) [16][21][25]. A separate hiring-manager (HM) call often repeats the deep dive and adds behavioral questions [9][22][23]
Shape	One project, taken apart: architecture, the decisions that were yours, trade-offs, measured impact. Interviewers keep asking "until they heard what they wanted to hear" [1][6][16]. In at least one loop the deep dive and a coding exercise shared a single session [26]. Behavioral and values questions are scored in separate rounds run by employees from any function [5]
Senior / L5 bar	**The perceived scope of your work is scored on its own.** Lead with problem size, ownership and outcome, not model details [1]. The round "also drives leveling" [2]. Scope judgements here can overturn strong technical rounds [2][3]
Weight	One recruiter told a candidate "the deep dive round is the most important", and Airbnb would not re-run it [18]. Several senior rejections were pinned on this round alone [8][18][20]
Reported failure modes	Opening walkthrough too long: keep it near 10 minutes, and one candidate's 20+ minutes ate the follow-up time [2]. A project too complex to explain or abstract quickly [8][14]. Generic answers without technical detail [18]. "No technical challenge visible" [10]. Background judged a poor match for the team [20]

Aspect	What sources report
MLE-specific twist	One MLE candidate's "ML Experience" round turned into an unexpected ML design question [7]. Prepare to whiteboard your project's system the way you would a design round
Prep advice from reports	Know every line on your resume: one candidate was grilled on a project that took one line [12]. Bring metrics, a conflict and a rejected alternative [2][3]. At staff level, compare your system with how Airbnb solved a similar problem on its tech blog, and say why you chose your approach [24]

How to use this chapter: choose **one** flagship ML project that touched an experiment or online metric, with clear cross-team reach. Practise a 10-minute arc, then drill every question below against that project. Prepare a backup project for questions that ask for a different story, such as a failure or a conflict.

Questions

A. Opening, project selection and the 10-minute arc

1. Walk me through the project you are proudest of. [Reported] [16][11][29]
    - Why this project and not another one on your resume?
    - Can you give me the two-minute version before we go deep?
2. Pick one recent project and walk me through it from start to finish. [Reported] [19] [25]
    - Where do you want to start: the problem, the system or the result?
3. Tell me about the most interesting project you have worked on. [Reported] [22]
4. Show me a project you are proud of that was cross-organizational or cross-functional. [Reported] [11]
    - Which teams outside yours had to change something for it to ship?
5. Walk me through the projects on your resume. What were you responsible for on each? [Reported] [13]
    - You wrote one line about this older project. Tell me about it in detail. [Reported] [12]
6. Tell me about the ML project with the biggest business impact that you led end to end. [Likely] [1]
    - Who used its output, and how many users, listings or dollars did it touch?
7. If you had to describe the project's scope in one sentence (problem size, your ownership, outcome), what would it be? [Likely] [1]
8. Can you draw the overall system on the whiteboard while you explain? [Reported] [14][6]

B. Problem framing and business context

9. What problem was the project solving, and why did it matter to the business at that time? [Reported] [17][6]
    - What would have happened if nobody had done this project?
10. What decision or product surface did the model's output feed into? [Likely] [1]
    - Who consumed the score: a ranker, a rule engine, an ops team or a user-facing UI?

11. How did you turn a vague product goal into a measurable prediction problem? [Likely] [1]
    - What was the label, and why that label rather than a nearby proxy?
12. What was the scale: traffic, QPS, data volume, number of entities? [Reported] [10]
    - What about that scale made the problem hard?
13. Why did you split the work into a short-term plan and a long-term plan? Why was the long-term plan needed at all? [Reported] [10]
14. Was ML actually necessary here? What was the heuristic or rules baseline? [Likely]
    - By how much did ML beat the baseline, and was the extra complexity worth it?
15. How did you size the opportunity before you started? [Likely]

C. Ownership: what was yours versus the team's

16. What was your specific role on the project? [Reported] [15][13]
    - Which decisions did you personally make, and which did you inherit?
17. How was the work divided across the team? [Reported] [9]
    - Who else was on the project, and what did each person own?
18. Did you lead the project, or contribute to it? How many people did you direct or influence? [Likely] [1]
19. Were the metrics you analysed already there, or did you add them? [Reported] [10]
20. What did you do that nobody asked you to do? [Likely]
21. If you had left halfway through, what would not have happened? [Likely]

D. Data, labels and features

22. Where did the training data come from, and what were its biggest quality problems? [Likely] [1]
    - How did you detect and handle label leakage?
    - How did you handle delayed or censored labels (for example a booking that happens weeks after a search)?
23. How did you build the label, and how noisy was it? [Likely] [1]
    - Did you consider regression versus a bucketed classification framing? [Likely] [1]
24. Which features mattered most, and how do you know? [Likely]
    - Did any "obviously useful" feature turn out to hurt?
25. How did you keep training and serving features consistent? [Likely]
    - Did you ever ship with training-serving skew? How was it caught?
26. How did you handle cold start for new users or new items? [Likely] [1]
27. Did you use spatial or geographic signals? How did you encode them? [Likely] [1]
28. Did your training data come from logs produced by an earlier model? How did you deal with feedback-loop and position bias? [Likely] [30]

E. Modeling decisions and trade-offs

29. Walk me through the architecture and the key design points. [Reported] [6][23]
    - Why this design over the obvious alternative?
30. What were the main trade-offs you made? [Reported] [9][21]
    - What did you give up, and how did you decide it was acceptable?
31. What alternatives did you consider and reject? [Reported] [2][3]

    - What evidence made you reject each one?
32. Why this model family (for example GBDT versus a neural network)? [Likely] [1][30]
    - When did you decide to move to something more complex, and what convinced you?
33. What was the single modeling change that gave the biggest gain? [Likely]
    - What did you try that did not work? [Likely] [30]
34. How did you tune the model, and how did you avoid overfitting to your validation set? [Likely]
35. How did you trade model quality against latency, cost or interpretability? [Likely]
36. If the interviewer cannot follow your domain, how would you simplify the core idea in two minutes? [Reported] [8][14]

F. Architecture, serving and production

37. Walk me through the implementation details end to end, from data pipeline to serving. [Reported] [6]
38. What were the latency and throughput requirements at serving time, and how did you meet them? [Likely]
39. Was inference batch, online or both? Why? [Likely]
40. How did you deploy new model versions safely (shadow, canary, rollback)? [Likely]
41. How did you make sure of system quality after deployment? [Reported] [17]
    - What monitoring and alerting did you set up for data drift and model decay?
42. What broke in production, and how did you find out? [Likely]
43. Which part of the system would break first at 10x scale? [Likely]
44. Did you build or reuse a platform (feature store, training pipeline, experiment framework)? Why? [Likely]

G. Offline-to-online evaluation and experimentation (A/B testing focus)

45. How did you evaluate the model offline, and which metric did you trust? [Likely] [1]
    - How well did the offline metric predict the online result?
46. How did an offline metric gain become an online metric gain? [Likely] [1]
47. How did you know it worked? Was there an A/B test? [Likely] [31]
    - What was the primary metric, and what were the guardrails?
    - What was the randomization unit, and why?
    - How long did it run, and how did you choose the sample size?
48. What were the confidence intervals on the main result? Were any segments surprising? [Likely]
49. Did offline gains ever fail to show up online? How did you debug it? [Likely] [30]
50. Did your experiment face interference effects (for example a two-sided marketplace or shared inventory)? How did you handle it? [Likely] [31]
51. Did you consider faster methods such as interleaving for a ranking change? Why or why not? [Likely] [32]
52. Did you use variance reduction such as CUPED? What did it buy you? [Likely]
53. How did you handle novelty effects, seasonality or ramp-up effects when you read the result? [Likely]
54. Did you check for sample ratio mismatch or logging bugs before you trusted the result? [Likely]

55. Did you keep a long-term holdout? What did it show compared with the initial test? [Likely]
56. Tell me about an experiment result that surprised you. What did you do next? [Likely]
57. Who made the launch decision, and what would have made you not launch? [Likely]

H. Impact and metrics

58. What was the measured impact, in business terms? [Reported] [2][3]
    - How did you attribute the impact to your work and not to other launches?
59. How did you ensure the users actually benefited after deployment? [Reported] [17]
60. What metrics did you use to define success, and who agreed to them? [Reported] [2]
    - How did you align Product and Data Science on the success metric? [Likely]
61. Were there negative side effects on another metric or another group (for example hosts versus guests)? [Likely]
62. If the impact was smaller than expected, why? [Likely]

I. Challenges, failures and debugging

63. What was the most challenging part of the project? [Reported] [16][15]
    - Was the challenge technical, organizational or about reliability?
64. What technical challenges did you hit, and how did you solve them? [Reported] [6][14]
    - Where exactly is the technical challenge in this project? [Reported] [10]
65. Tell me about a project or plan that missed its ETA, or whose scope had to shrink. [Reported] [9]
    - How did you identify the root cause? [Reported] [9]
    - Did you have a fallback plan to unblock the team? [Reported] [9]
66. Tell me about a model that worked offline but failed in production. [Likely]
67. Tell me about a bug or data problem that invalidated results. How was it found? [Likely]
68. What was the hardest debugging problem on the project? [Likely]
69. Tell me about a failure on this project and what you did next. [Reported] [3][4]

J. Planning, milestones, collaboration and conflict

70. How did you plan the project? [Reported] [15]
    - How did you set and deliver milestones? [Reported] [17][11]
71. How was the engineering effort structured? [Reported] [11][29]
72. How did you persuade stakeholders to back the project? [Reported] [2]
    - Who opposed it, and what changed their mind?
73. Did you have a conflict on the project? How did you resolve it? [Reported] [15][16][9]
    - What would the other person say about how it was resolved?
74. How did you work with your teammates? [Reported] [16]
75. Tell me about collaborating with another team where their delay affected you. [Reported] [9]
76. How did you keep teams in different time zones or cultures aligned from design through launch? [Reported] [9]
77. How did you handle a tight deadline or a complex project you owned? [Reported] [4]
78. How did you work with data scientists on metric definitions and experiment reading? [Likely]

79. How did you influence the technical direction without formal authority? [Likely]

K. Retrospective, learning and next steps

80. If you redid the project, what would you change? [Reported] [15][17][3]
    - What would you change if you had to redesign it from scratch? [Reported] [17]
81. What did you learn from the project? [Reported] [16]
82. What would the next iteration of the system look like? [Likely]
83. If you had half the time, what would you cut? [Likely]
84. What technical debt did you leave behind, and why was it acceptable? [Likely]
85. What was the biggest obstacle in moving from writing code to owning a whole project? [Reported] [9]

L. Leadership and senior/staff calibration

86. Walk me through your experience, a past project and your leadership on it. [Reported] [21]
87. How did you grow or mentor others on this project? [Likely]
88. What is your current scope at work, and what scope are you looking for? [Reported] [10]
    - Are you looking to be a tech lead, or a strong IC? [Reported] [10]
89. How did this project change how your team or organization works? [Likely]
90. Which decision on this project would you defend most strongly to a skeptical staff engineer? [Likely]
91. How did you decide what not to build? [Likely]

M. Mapping your project to Airbnb

92. How does your system compare with how Airbnb solved a similar problem, and why did you choose your approach? [Reported] [24]
    - Have you read Airbnb's engineering blog posts on search ranking, pricing or experimentation? How does your work differ? [Likely] [24][30][31]
93. How would your project transfer to a two-sided marketplace like Airbnb's (guests and hosts)? [Likely]
94. What in your background maps directly onto this team's problems? [Reported] [20][2]
    - You have also worked in an unrelated area (for example hardware). How is that relevant here? [Reported] [20]
95. If you joined, what would you build in your first six months, based on this experience? [Likely]
96. Tell me about a project connected to your personal or professional values. How did it embody or challenge them? [Reported] [19][33]

N. Pressure probes reported in the room

These are the push-back styles candidates describe. Practise staying concise and concrete under each one.

97. "I still don't understand. Can you explain the task again, more simply?" [Reported] [14]

98. "Why did you do that at all?" (repeated challenges to the rationale for a phase of the project) [Reported] [10]
99. "Can you be more specific? What were the actual technical details?" (asked because answers sounded generic) [Reported] [18]
100. Interruptions and nitpicking mid-answer: how do you regain the thread? [Reported] [27]
101. "Let's switch: how would you design an ML system for X?" (the experience round turns into ML design) [Reported] [7]
102. "Give me the specific API and database design of your system." (detail-level probe rather than scaling talk) [Reported] [28]

References

[1]-[29] were fetched and read for this chapter. [6]-[29] are English or Chinese 1point3acres posts mirrored on GitHub. The mirror does not keep original post dates, though several posts carry 2025 timestamps. [30]-[33] were **not** fetched and are cited from prior knowledge. Verify them before relying on them.

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview, CC BY 4.0; based on 3 candidate reports, 2026 Q2). GitHub mirror kevin-2023-code/Tech-Interview-Questions, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb Software Engineer Interview Process" (TrueInterview, CC BY 4.0; 41 candidate reports, 2025 Q4 to 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
3. "Airbnb interview process" (TrueInterview general guide). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. "Airbnb" company guide (Technical Experience round, behavioral list; updated October 2026). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
5. "Interview formats of top companies" (Airbnb section). Tech Interview Handbook, GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
6. 1p3a post #40 (MLE loop): project deep dive covering architecture, implementation, challenges and solutions; ML design on LTV and host pricing. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
7. 1p3a post #79 (MLE loop): "ML Experience" round expected as a project deep dive but turned into an ML design question; two ML design rounds. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
8. 1p3a post #10 (senior, rejected on design and resume-project rounds): 1-hour round ran 90 min on one project; advice to keep it under 20 min and simplify. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/10.md
9. 1p3a post #11: VO3 "Tech Experience Interview" (45 min); HM round with deep dive plus BQ (missed ETA or reduced scope, root cause, division of work, fallback plan,

cross-timezone collaboration); experience round "trade-offs, conflicts". https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/11.md

10. 1p3a post #13 (senior): HM scope call (TL or IC); project deep dive challenged on the long-term plan, traffic level, new versus existing metrics, and the lack of a technical challenge. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/13.md
11. 1p3a post #8: "project experience: most proud project, milestone, eng structure, conflict"; "BQ: show a proud project (cross organizational/functional)". https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/8.md
12. 1p3a post #3: experience BQ round asked about a project described in a single resume line. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/3.md
13. 1p3a post #30: phone round with a 30-minute resume deep dive on responsibilities. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/30.md
14. 1p3a post #39: 1-hour resume round felt like a stress interview ("couldn't understand"); advice to focus on role, decisions, technical challenges and conflicts. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/39.md
15. 1p3a post #58: project deep dive covering role, planning, challenges, conflicts and redo improvements. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/58.md
16. 1p3a post #69: Technical Experience Interview covering proudest project, most challenging part, teammates, conflicts and lessons. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/69.md
17. 1p3a post #84 (senior backend): project presentation covering scope, milestones, post-deploy quality and user benefit, and redesign. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/84.md
18. 1p3a post #86: failed the deep dive for "generic answers without any technical details"; recruiter said the deep dive is the most important round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/86.md
19. 1p3a post #93: experience round as a deep dive into a recent project; values-linked project prompt. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/93.md
20. 1p3a post #7: deep-dive feedback said the candidate's background did not fit the team. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/7.md
21. 1p3a post #78 (staff SWE, L6): senior manager from another group ran experience, past projects and leadership. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/78.md
22. 1p3a post #29 (senior): HM went over the most interesting project in detail; VO behavior round focused on one project. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/29.md
23. 1p3a post #62 (infrastructure senior): HM interview on resume projects, key design points and challenges. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/62.md.md
24. 1p3a post #94 (staff): HM round deep on the resume; advice to read Airbnb's tech blog and compare your system with Airbnb's. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/94.md

25. 1p3a post #99: experience interview "dug into a project from start to finish". https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/99.md.md
26. 1p3a post #104: project deep dive and coding combined in one round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/104.md
27. 1p3a post #31: a manager in the behavioral and in-depth project round kept interrupting and nitpicking. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/31.md
28. 1p3a post #6: "BQ and past experience deep dive" round; interviewers wanted API and database detail. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/6.md
29. 1p3a post #18: "most proud project, milestone, eng structure, conflict", asked in fine detail. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/18.md
30. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD 2019 (includes lessons on failed attempts and offline/online gaps). Not fetched. https://arxiv.org/abs/1810.09591
31. "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014. Not fetched. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
32. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Tech Blog (Medium), 2022. Not fetched. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
33. "Life at Airbnb" (core values), Airbnb Careers. Linked from [4]; not fetched. https://careers.airbnb.com/life-at-airbnb/

08. Core Values Interviews: Champion the Mission, Be a Host, Embrace the Adventure, Be a Cereal Entrepreneur, plus Cross-Functional and Leadership Behavioral Questions

> Research note (read first). The shared web-search budget ran out before this chapter was written, so no new search-engine queries were possible. DNS lookups also failed for glassdoor.com, leetcode.com, interviewing.io, hellointerview.com, prepfully.com, igotanoffer.com, medium.com, careers.airbnb.com and news.airbnb.com, and reddit.com refused fetches. The chapter is therefore built from pages that could be read directly on GitHub. These include two TrueInterview-derived Airbnb process guides (one for the MLE track, one for the SWE track), a 2026 company guide that aggregates Hello Interview, interviewing.io, PracHub and LeetCode Discuss, the Tech Interview Handbook's Glassdoor-sourced Airbnb list, and a mirror of more than 100 1point3acres (1p3a) Airbnb interview posts. Every 1p3a post was downloaded and searched for values, behavioral (BQ), hiring-manager (HM) and culture content. Chinese-language posts are paraphrased in English here. Tags: **[Reported]** means a cited source says a candidate was asked this, or something very close to it, at Airbnb. **[Likely]** means an inference from the values, the Senior MLE / experimentation role, or Airbnb's products. It does not mean anyone reported it.

Format notes

Aspect	What sources report
Number and length	One core-values round of about 45 minutes is reported for the MLE track [1]. The SWE track reports one or two rounds of about 45 minutes each [2][3]. Hello Interview's G8 guide (as aggregated) calls it a "30-min mandatory gatekeeper" [4]. Rounds sometimes end early: one 2026 candidate finished in about 20 minutes and still passed [4][10]
Timing	Often scheduled **after** the technical onsite has passed, as a separate gate [4][8][10]. One candidate had it after hiring-manager calls during team matching [18]
Interviewer	An employee from **any function**, outside the hiring team, often a non-engineer [1][2][4][5][7]. One senior candidate's values round was with an engineering manager [15]
What is scored	Authentic connection to Airbnb's four values: Champion the Mission, Be a Host, Embrace the Adventure, Be a "Cereal" Entrepreneur [5][12]. Personal stories count as much as work stories [2]. Answers that relabel engineering wins with value names, or that sound rehearsed or framework-driven, land poorly [1][2][3]. Rapport with the interviewer affects the outcome [1]

Aspect	What sources report
Weight	Scored in the same debrief as the technical rounds [2]. It "can reject you on its own" [4][5]. One candidate failed two behavioral rounds and was told they "answered too quickly, without their own pace" [16]
Related rounds	**Experience / project deep dive** (45-60 min): cross-functional influence, conflict, milestones and "who outside your team you had to align" are probed here. It drives leveling [1][2][17][18][19]. **Hiring manager conversation**: scope, past projects, team fit, and sometimes BQ questions about missed cross-team ETAs [1][8][20]
Senior bar	No source defines a separate senior bar for the values round itself [1][2]. For Senior, scope, cross-team influence and ownership are judged mainly in the deep dive and HM rounds [1][2][20]. Expect values interviewers to probe for the same signals: mentoring, hosting newcomers, and leading through ambiguity [Likely]

Preparation signals from sources: prepare one or two stories per value, including non-work ones such as a cause you support, a time you hosted or built community, and feedback that stung [2][3][4]. Use plain language for a non-engineer interviewer [5]. Pace your answers and do not rush [16].

Questions

A. Opening, background and "Why Airbnb?"

1. Walk me through your background. [Reported] [8]
    - What thread connects your career moves?
    - What do you do outside work?
2. Why Airbnb? [Reported] [1][3][4][6][8][11]
    - Why now, and why this team rather than a bigger-name ML org?
    - What specifically drew you to Airbnb as a company, beyond the product? [Reported] [1]
3. What do you think of Airbnb? [Reported] [6]
    - What does Airbnb do better than any other travel company?
4. What excites you about the company? [Reported] [6]
5. Tell me something about yourself and why you would be a good fit. [Reported] [6]
6. What are your hobbies outside work, or what fun thing from school would you share? [Reported] [8]
7. Is there anything you really want to do in the future? [Reported] [4]
    - Where do you want to be in five years, and how does Airbnb help?
8. What is something you do not want to repeat from your last job? [Reported] [6]
9. What obstacles, challenges or uncertainty do you expect if you join Airbnb? [Reported] [8]
    - What would make you leave Airbnb after a year?

B. Champion the Mission (belonging, community, causes)

10. What does "belong anywhere" mean to you? [Reported] [6]
    - When have you felt you did not belong, and what changed it?
    - When have you created belonging for someone else?
11. What part of our mission resonates most with you? [Reported] [6]

12. How does Airbnb impact our guests and hosts? [Reported] [6]
    - How does it affect the neighborhoods and cities where hosts operate?
13. What large problems in the world would you solve today? [Reported] [1][6]
    - Why that problem, and what have you personally done about it?
14. What cause do you actively support, and how do you contribute to it? [Reported] [1] [3]
    - How much time do you give it, and what have you learned from it?
15. When did you last volunteer, and what did you take away from it? [Reported] [8]
16. How have you contributed to a community, or what volunteer work have you done? [Reported] [4][5]
17. Tell me a story about your community and the connections you have built. [Reported] [11]
18. Describe a time you stood up for something you believe in. [Reported] [4]
    - What did it cost you, and would you do it again?
19. Who has inspired you most, and why? [Reported] [11]
20. What is Airbnb's hardest problem today, in your view? [Reported] [2]
    - How could ML or experimentation help solve it?
21. Tell me about a project that reflected your personal or professional values. [Reported] [14]
    - How did the project embody or challenge those values?
22. Name an organization (not Airbnb) that you admire, and explain why. [Reported] [4] (intern prompt, flagged unverified by the source)
23. Tell me about a time you chose the right outcome for users even when an internal metric pointed the other way. [Likely]
    - In ML terms: when did you decline to ship a model that won the A/B test?
24. How would you make sure a ranking or pricing model treats hosts and guests fairly across demographics and markets? [Likely]
    - How would you explain that trade-off to a non-technical interviewer?
25. How do you keep the people affected by your models (hosts, guests, support agents) in mind when you only see aggregate metrics? [Likely]

C. Be a Host (care for guests, teammates and newcomers)

26. Give me an example of when you have been a good host. [Reported] [2][6]
    - It does not have to be a listing: an event, a dinner, a visitor, a new hire? [Reported] [11]
27. Do you have any experience as a host, of any kind? [Reported] [11]
28. Tell me about your most memorable experience traveling with Airbnb. [Reported] [8]
    - What did the host do that made it memorable?
29. What are your tips for choosing an Airbnb listing? Have you ever had a bad experience? [Reported] [8]
    - How would you fix that bad experience with product or ML?
30. Share one of your trips with us. / Tell me about your most memorable trip. [Reported] [4][6]
31. Name a situation where a company's customer service impressed you. [Reported] [6]
    - What would it take for Airbnb support to match it?
32. What is the best gift you have ever given or received? [Reported] [6]

33. If you had an unlimited budget to buy one gift for one person, what would you buy and for whom? [Reported] [6]
34. Was there a senior colleague who helped you a lot early in your career? [Reported] [8]
    - How have you paid that forward?
35. Tell me about a time you made a new teammate feel welcome and productive. [Likely]
    - What did you change in onboarding docs, tooling or pairing as a result?
36. Describe a time you mentored an engineer or scientist to a measurable improvement. [Likely]
    - How did you adapt your approach to their style?
37. Tell me about a time you went out of your way for an internal customer, such as a data scientist or product team that consumed your model or platform. [Likely]
38. Tell me about a time you supported a teammate going through a hard period. [Likely]
39. How do you make people who disagree with you feel heard in a design review or experiment readout? [Likely]
40. Tell me about a time you had to give someone terrible news. [Reported] [5][6]
    - How did you prepare, and how did they react?

D. Embrace the Adventure (curiosity, risk, discomfort, resilience)

41. What adventurous moments or decisions have you had in life? How do you take risks? [Reported] [8]
42. Tell me about a calculated risk you took. [Reported] [2]
    - How did you size the downside before you committed?
43. Tell me about a time you were uncomfortable and how you dealt with it. [Reported] [5][6]
44. If you had an unlimited budget and could go anywhere, where would you go? [Reported] [6]
45. What is something new you can teach your interviewer in a few minutes? [Reported] [6]
46. Tell me about a self-learning experience: how you learned something, and whether it helped your later work. [Reported] [13]
47. What is the most challenging project, in or out of work, that you have done recently? [Reported] [6][11]
48. How do you manage the gap between expectations and reality, at work or in life? [Reported] [8]
49. Tell me about a time a major change you did not choose (reorg, layoff, pivot) hit your team. How did you respond? [Likely]
50. Tell me about a time you worked in a domain you knew nothing about, for example moving from recommendation to causal inference. [Likely]
    - How quickly were you useful, and what did you get wrong first?
51. Tell me about an experiment or model idea that failed. What did you learn, and what did you do next? [Likely]
    - How did you tell the team?
52. When have you changed your mind because of data that contradicted a strong belief? [Likely]
53. If you were a gerbil, which gerbil would you be? (a reported curveball used to test spontaneity) [Reported] [6]

E. Be a "Cereal" Entrepreneur (resourcefulness, scrappiness, creativity)

54. Tell me about a time you were scrappy and built something valuable with very limited resources. [Likely] [5][12]
    - What did you deliberately leave out?
55. What is something you would like to remove from the Airbnb experience? [Reported] [5][6]
    - How would you test whether removing it helps?
56. Tell me about a time you saw an unowned problem and took it on without being asked. [Likely]
57. Describe a time you shipped a quick heuristic or a small experiment instead of waiting for the perfect model. [Likely]
    - How did you know when it was time to replace it?
58. Tell me about a time you found a creative workaround for a data, compute or labeling constraint. [Likely]
59. Have you ever started something of your own, such as a side project, club, startup or open-source tool? [Likely]
    - What did it teach you about customers?
60. Tell me about a time you turned a one-off analysis or script into a tool other teams adopted. [Likely]
61. Tell me about a time you reduced cost (infrastructure, experiment duration, labeling spend) without hurting quality. [Likely]
    - Which metrics showed it, and who signed off?
62. Pitch me one product or ML idea for Airbnb that nobody has asked you to build. [Likely]
    - How would you validate it in two weeks?

F. Feedback, failure and self-reflection

63. Have you given or received hard feedback? Tell me about it. [Reported] [2][3][11]
    - What did you change afterwards, and how would the other person describe it?
64. Have you received negative feedback from others, and how did you handle it? [Reported] [13]
65. Tell me about a time you did not meet expectations, and how you improved using customer feedback. [Reported] [13]
66. What could you have done better in a past experience? [Reported] [8]
67. What was your most difficult task, and how did you get through it? [Reported] [11]
68. Tell me about a time you failed. [Likely] [3]
    - What would you do differently, and what did you change in your process?
69. What would your last manager say is your biggest area for growth? [Likely]
70. Tell me about a time you made a mistake that affected users or a launch, such as a bad model push or a wrong experiment readout. [Likely]
    - How did you communicate it, and what guardrail did you add?

G. Cross-functional collaboration and communication

71. Give an example of communicating with your own team or with a different team. [Reported] [8]

72. How do you make sure a design lands well with teams in different time zones or cultural backgrounds? What problems came up? [Reported] [8]
73. In an urgent situation, how do you make sure you get time with senior people? [Reported] [8]
74. How did you work with senior management on large projects and with multiple internal teams? [Reported] [6]
75. Tell me about a time another team's internal ETA slipped and affected you, so the timeline moved or scope shrank. [Reported] [8]
    - How did you identify the root cause? [Reported] [8]
    - How was work split across the team? [Reported] [8]
    - Did you have a backup plan to unblock yourself? [Reported] [8]
76. Show me a project you are proud of that crossed organizations or functions. [Reported] [17]
    - How were milestones set, how was engineering structured, and what conflicts arose? [Reported] [17]
77. Tell me about a cross-team collaboration challenge on a past project. [Reported] [18]
78. Tell me about a conflict, failure, tight deadline or complex project you owned. [Reported] [4]
79. How did you set milestones on your project, and how did you ensure system quality and user benefit after deployment? [Reported] [19]
    - What would you change if you redesigned it? [Reported] [19]
80. Tell me about a time you disagreed with a product manager about the success metric for an experiment. [Likely]
    - Whose metric won, and why?
81. Tell me about a time a data scientist and you read the same A/B test differently. How was it resolved? [Likely]
82. How do you explain a model's behavior or an experiment's uncertainty to a non-technical stakeholder, such as an operations or policy partner? [Likely]
    - Explain a confidence interval to me as if I were your values interviewer.
83. Tell me about a time you influenced a decision with no formal authority over the people making it. [Likely]
84. Tell me about a time you had to say no to a partner team's request. [Likely]
85. How do you work with designers and PMs when a model's output changes the user experience, for example in ranking or pricing suggestions? [Likely]
86. Describe a time you resolved friction between two teammates or two teams. [Likely]

H. Senior-level ownership and leadership (MLE and experimentation flavor)

87. What is the biggest obstacle in moving from writing code to owning an entire project? [Reported] [8]
88. What is your current scope, and what scope do you expect here? [Reported] [20]
    - Do you want to be a tech lead, or a strong IC on a team that already has one?
89. What was the hardest technical or organizational challenge you faced? [Reported] [20]
90. Tell me about a time you set technical direction for a group larger than your own team. [Likely]

91. Tell me about a time you stopped a launch, or argued against shipping, because the experiment evidence was weak. [Likely]
    - How did leadership react, and what happened next?
92. Tell me about a time you raised the bar for experimentation rigor on your team, for example with pre-registration, guardrails or a review process. [Likely]
93. How do you balance short-term booking or revenue metrics against long-term guest and host trust? [Likely]
94. Tell me about a time you had to deliver an inconclusive or negative result to a team that badly wanted a win. [Likely]
95. How do you grow other engineers' judgment about when ML is the wrong tool? [Likely]
96. Tell me about a time you made a decision with incomplete data under time pressure. [Likely]
97. How do you prioritize when three teams all need your help this quarter? [Likely]
98. Tell me about a time you handed off something you built so that it no longer depended on you. [Likely]

I. Hiring-manager conversation (behavioral portion)

99. Walk me through your most interesting project. What were the key design points and challenges? [Reported] [20][21]
100. Which problems on our roadmap map onto your background, and where would you need to ramp up? [Reported] [1]
101. Who outside your team did you have to align on your biggest project, and how? [Reported] [1][2]
102. How did you scope work when requirements changed mid-project? [Reported] [2]
103. Why did you choose a long-term plan over a short-term fix, and how did you justify it? [Reported] [20]
104. Which metrics did you add yourself, and which existed already? [Reported] [20]
105. What do you need from a manager to do your best work? [Likely]
106. What kind of team culture do you create when you are the most senior IC in the room? [Likely]

References

All URLs below were fetched or downloaded for this chapter, except where noted. The 1p3a mirror posts are English translations or original Chinese posts reposted on GitHub. The original 1point3acres threads were not reachable.

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview guide, mirrored; based on 3 candidate reports, mainly 2026 Q2). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb Software Engineer Interview Process" (TrueInterview guide, mirrored; based on 41 candidate reports, 2025 Q4 to 2026 Q2). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md

3. "Airbnb Interview Process" (overview). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. "Airbnb" company guide (aggregates Hello Interview G8 guide, interviewing.io, PracHub Jan 2026, LeetCode Discuss post 7621156 from Mar 2026, Glassdoor intern snippets). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
5. "Behavioral: other companies" (Airbnb section: values, format, practice questions). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/behavioral/other-companies.md
6. "Behavioral interview questions" (Airbnb list, sourced from Glassdoor). Tech Interview Handbook (yangshun), GitHub, undated (older SWE reports). https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/behavioral-interview-questions.md
7. "Interview formats of top companies" (Airbnb: two cross-functional rounds with employees from any discipline). Tech Interview Handbook (yangshun), GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
8. 1p3a post #11 (two posts in one file: a core values round opening with "When did you last volunteer?", and a 4.5 YOE candidate's full VO with HM BQ and a 15-question Core Values list). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/11.md
9. Airbnb 1point3acres interview-post mirror (folder index). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/tree/main/company%20wise/airbnb/1p3a
10. 1p3a post #3 (passed onsite and core values; values round scheduled after onsite and ended in about 20 minutes; posted 2026-03). FlowSight/coding, GitHub, 2026. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/3.md
11. 1p3a post #57 (core values questions: why Airbnb, hosting experience, hard feedback, community, who inspired you, most difficult task). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/57.md
12. 1p3a post #55 (G10 full-stack; core value round "essentially behavior" demonstrating hospitality and entrepreneurial spirit; 2025-07). FlowSight/coding, GitHub, 2025. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/55.md
13. 1p3a post #45 (BQ opening: self-learning, missed expectations and customer feedback, negative feedback). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/45.md
14. 1p3a post #93 (core value round listed; project tied to personal or professional values). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/93.md
15. 1p3a post #75 (core value interview scheduled with an engineering manager). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/75.md
16. 1p3a post #7 (onsite with culture and behavior rounds; failed both BQ rounds for answering too fast). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/7.md
17. 1p3a post #8 (BQ: proud cross-organizational project; milestones, eng structure, conflict). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/c

ompany%20wise/airbnb/1p3a/8.md
18. 1p3a post #28 (experience round with cross-team collaboration questions; HM calls, then core value round; 2025-10). FlowSight/coding, GitHub, 2025. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/28.md
19. 1p3a post #84 (senior backend; project discussion on milestones, quality and user benefit after deployment, redesign). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/84.md
20. 1p3a post #13 (HM round on current scope and TL versus IC expectations; deep dive challenged long-term plan, metrics and technical challenge). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/13.md
21. 1p3a post #29 (Senior; HM interview with behavioral questions on the most interesting project). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/29.md